

ECE 4760 Lab 1: Bird Song Synthesizer

By David Mayorga (dsm283), Joanna Saint-Pierre (jes624), Rawsen Weideman (rlw332)

Introduction

In this lab, we designed and constructed a digital bird-song synthesizer using a Raspberry Pi Pico2 microcontroller interfaced with an MCP4822 digital-to-analog converter (DAC). Upon compiling and running the code, the synthesizer outputs a continuous tone whose frequency can be adjusted by moving a slide potentiometer back and forth along its axis. Depending on what keys the user inputs on a 4x3 keypad, the user can utilize the synthesizer in several ways, including being able to completely silence or unmute its output (pressing the “0” key), record various unique short-duration frequency sweeps and save them to the keypad on keys “1” through “9”, play back (accelerated) recorded frequency sweeps, and even arrange previously-recorded sweeps into specific sequences to be played back as a continuous composition after a single key press. Through the implementation of the code and construction of the synthesizer in both hardware and software, the end goal of this lab is to use the synthesizer to successfully be able to reproduce the chirps, pauses, and short swoops of the Northern Cardinal bird, as well as other birds, closely enough that Merlin, the Cornell Lab of Ornithology's bird identification application, can recognize the output as that of an actual bird while allowing us to explore fundamental concepts of embedded systems such as timer service interrupts, analog-to-digital conversion, serial communication, and direct digital synthesis.

Design and Testing

1. Hardware

1.1 General Overview

The hardware implementation of the digital bird-song synthesizer largely revolved around the use of the Raspberry Pi Pico2 microcontroller as the central component of the device. The microcontroller interfaced with the external peripherals on the board to monitor user inputs, transform them into state machine logic and ADC values after performing DDS calculations on the waveforms, and communicate with an on-board digital-to-analog converter (DAC) such that the resulting audio waves could be output to a speaker.

These external peripherals feature a 4x3 keypad, a 4219_C11375 slide potentiometer, an MCP4822 Digital-to-Analog Converter (DAC), and a Raspberry Pi Debugger Probe. Together, the keypad and slide potentiometer allow the user to choose between the various operating modes and input frequencies sent to the Raspberry Pi Pico2 microcontroller, while the DAC receives the transmitted DDS digital audio samples from the Raspberry Pi Pico2 microcontroller through timer-driven interrupts over SPI communication and converts these samples into an analog voltage signal that can produce the desired audio output. The Raspberry Pi Debugger Probe is not directly necessary to include in the hardware for the functionality of the synthesizer, but it provides a separate UART interface through GPIO pins 0 and 1 for serial data transmission and reception, which can be helpful for debugging. All connections between the hardware components can be seen in the two diagrams below, with a high-level pin-mapping diagram of the complete synthesizer circuit shown first in Figure 1 and the actual synthesizer circuit being shown in Figure 2.

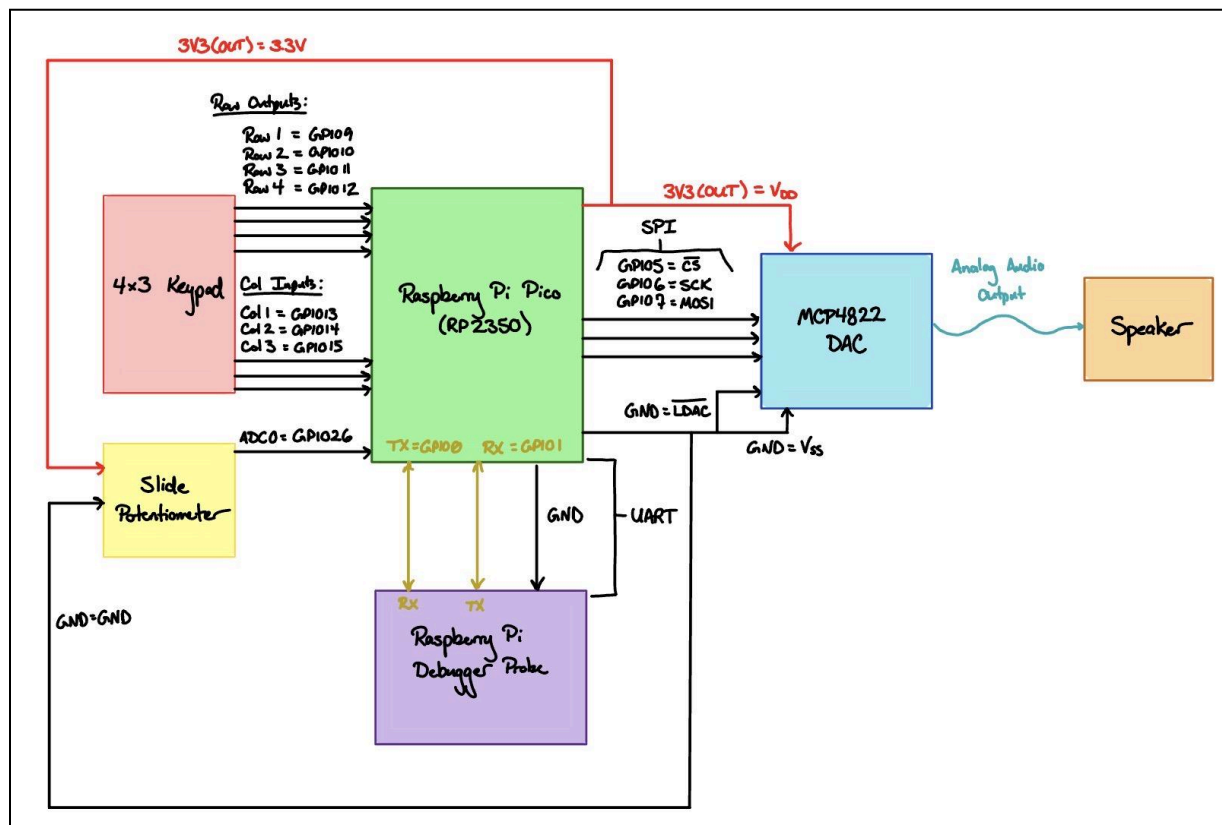


Figure 1: A high-level pin-mapping diagram of the complete synthesizer circuit. Arrows indicate the direction of the connection, while the layout models a left-to-right input to output structure.

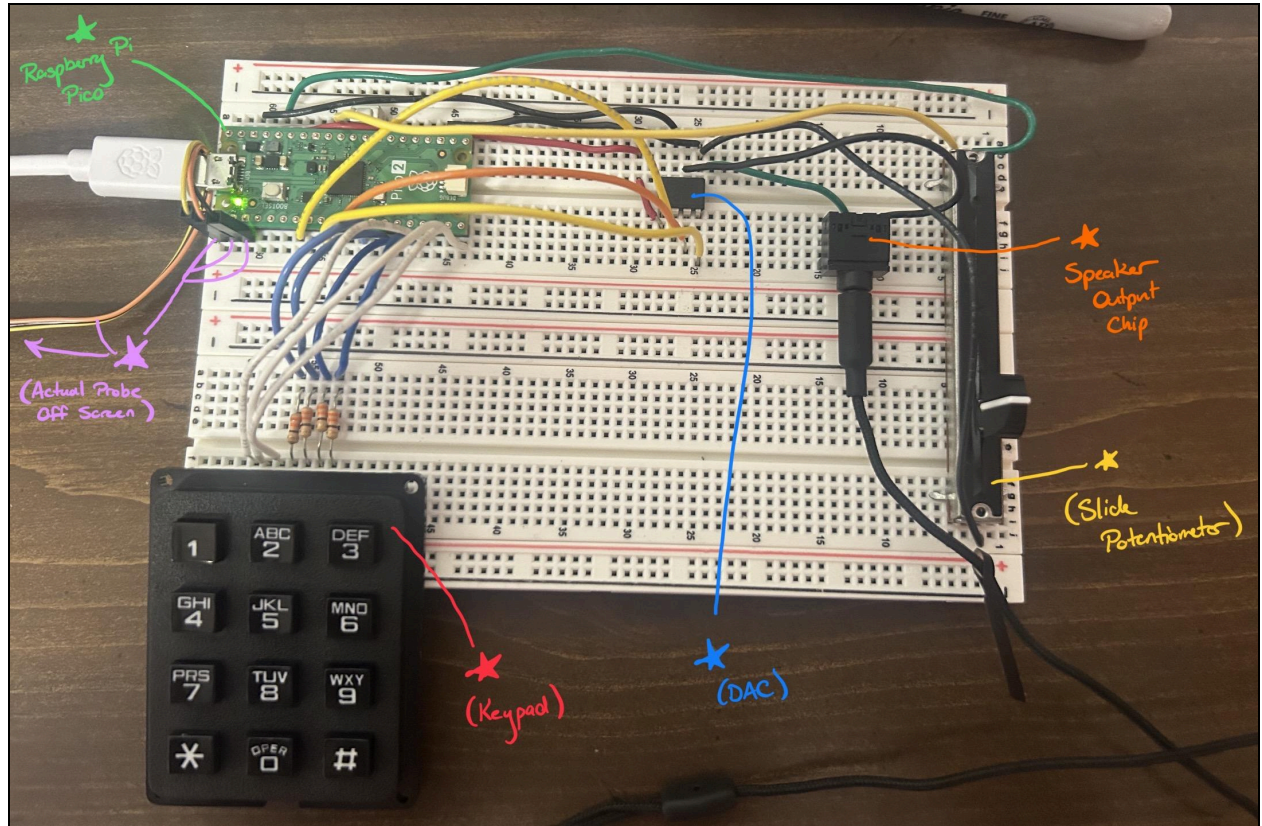


Figure 2: The complete bird-song synthesizer circuit. The above figure has been annotated such that hardware components are easily identifiable and consistent with the color labels given to them in Figure 1.

1.2 Raspberry Pi Pico2 Microcontroller

The Raspberry Pi Pico2 microcontroller serves as the central processing unit of the digital bird-song synthesizer and is responsible for executing state machine logic based on keys inputted onto the keypad via shorted pull-down GPIO input-output connections, the DDS algorithm based on the positioning of the potentiometer, and communication with the DAC over SPI. The way in which the microcontroller can interface with each of the external peripheral components of the synthesizer is through the use of various GPIO pins, which can detect the various keypad inputs and changes in the potentiometer's position that affect the operational mode and input analog voltages sent to the DDS algorithm at any given time. Through GPIOs 5, 6, and 7 specifically, the microcontroller is able to communicate with the DAC using its internal analog-to-digital converter (ADC), which samples and converts analog voltage inputs from the potentiometer into digital values used to determine the frequency of the generated waveform. After applying the

DDS algorithm to these frequencies, digital waveform samples are generated and transmitted to the DAC, which converts the samples back to analog voltages and plays them out on the speaker (so long as the program is not currently muted).

The internal system clock of the microcontroller is configured to operate at 150 MHz, which allows ample processing speed for both on-board waveform generation and off-board user input operations. However, it is important to note that not everything within the microcontroller operates at this frequency, as there is a hardware timer configured to trigger an interrupt service routine (ISR) at a differing frequency of 50 kHz (and thus a sampling interval of 20 μ s). This difference in frequency is crucial for allowing the microcontroller to maintain its keypad scanning, recording, and playback operations to continue while providing a consistent audio sampling rate for each interrupt to update the DDS phase accumulator, and subsequently cause the sine table to retrieve a new corresponding value which is sent as a new digital waveform sample to the DAC.

1.3 MCP4822 DAC

The MCP4822 is a dual-channel, 12-bit digital-to-analog converter (DAC) which is responsible for converting the digital waveform samples generated by the microcontroller (after it applies the DDS algorithm) to an analog voltage signal which can be played out on a speaker. Given the ISR that the microcontroller abides to, a new digital waveform sample is able to be sent to the DAC every 20 μ s (following the microcontroller updating its DDS accumulator and using a 256-entry sine lookup table to find the corresponding waveform sample to transform into a 12-bit digital output sample) over SPI communication. As the SPI interface is configured to a clock frequency of 20 MHz using 16-bit data transfers, a digital sample from the microcontroller is thus transmitted to the DAC in 0.8 μ s, which allows plenty of time for remaining DDS computations and other program operations to perform.

The connections made with GPIOs 5, 6, and 7 between the microcontroller and the DAC can be found in the table below (Table 1). In our implementation, DAC channel B is used to generate the analog output signal (we could have kept it on channel A, but we decided keeping channel B would best demonstrate adherence to the Week 1 checkpoint).

Microcontroller-DAC Connection	What the Connection Does
3V3(OUT) — VDD	Supplies power to the DAC.
GND — VSS, LDAC	Supplies common GND, LDAC isn't used in this lab.
GPIO5 — CS (Chip Select)	Controls the beginning and end of SPI connections.
GPIO6 — SCK (Clock)	Provides the clock signal for serial data transmission.
GPIO7 — MOSI (Main In, Sec Out)	Transmits digital waveform samples to the DAC.

Table 1: A table outlining the connections between the Raspberry Pi Pico2 and the DAC.

1.4 Keypad

The 12-button keypad serves as one of two primary user interfaces of the digital bird-song synthesizer and allows the user to control tone generation, recording, playback, and composition through specific combinations of button presses, holds, and releases of differing (groups of) keys. The keypad is arranged in a 4x3 (four-row, three-column) matrix, allowing the microcontroller to identify all twelve buttons using just seven GPIO connections rather than requiring an individual GPIO pin for each button. The rows are connected as outputs to GPIOs 9-12 through 330Ω resistors, while the columns are connected to GPIOs 13-15 and are configured as inputs with internal pull-up resistors (of roughly 500,000Ω). This difference between impedance values is mainly a result of getting a strong determination of a signal being either 0 or 1, as resistors closer in magnitude would resemble a voltage divider circuit that splits the 3.3V input voltage more evenly, thus leaving ambiguity as to whether a connection is a logic 0 or logic 1 connection. Therefore, by sequentially driving each row and monitoring the corresponding column input GPIOs, the microcontroller is able to identify which of the 12 buttons is currently being pressed based on its unique row-column combination being pulled down and shorted. A table detailing the connections between the keypad and the microcontroller can be found below in Table 2.

Keypad Connection	Microcontroller Connection
Column 1	GPIO 13
Column 2	GPIO 14

Column 3	GPIO 15
Row 1	GPIO 9
Row 2	GPIO 10
Row 3	GPIO 11
Row 4	GPIO 12

Table 2: A table outlining the connections between the Keypad and the Raspberry Pi Pico2.

1.5 Slide Potentiometer and Digital-to-Analog Conversion

The slide potentiometer is the second of the two primary user interfaces of the digital bird-song synthesizer and controls the frequency of the sinusoidal waveform generated by the device. In principle, the slide potentiometer operates as a variable voltage divider, which produces an analog voltage that changes according to the position of the slider along its axis. This ever-changing voltage is connected to GPIO 26 of the microcontroller, which serves as an analog input to the microcontroller's internal analog-to-digital converter (ADC). Sampling this incoming voltage allows for an ADC-corresponding digital value to be generated, and the microcontroller can use this value to determine the desired waveform frequency (which is then used by the DDS algorithm to calculate the phase increment required to generate the desired sinusoidal waveform). In our implementation, this frequency is calculated by multiplying the ADC reading by a scaling factor of 2.5, as the slide potentiometer's max output is 4095 needed to be scaled up to produce frequencies at around the roughly 10k Hz range specified for the lab.

During standard tone generation, adjusting the potentiometer changes the frequency of the analog signal produced by the DAC, while during recording, the microcontroller periodically samples and stores these frequency values in memory for subsequent playback. Given the program's recording interval of 10 ms, the microcontroller is able to store approximately 100 frequency samples per second, with each of the nine available numeric keys capable of storing up to 1,000 samples (corresponding to a maximum recording duration of approximately 10 seconds). During playback, these recorded frequency values are retrieved at approximately 1 ms intervals, allowing the original frequency sweeps to be reproduced at approximately ten times their recorded speed.

1.6 Serial Connection and Debugger Probe

When debugging over UART, the Raspberry Pi Debugger Probe serves as the primary interface between the digital bird-song synthesizer and an external computer, allowing the program to be uploaded to the microcontroller and its internal operations to be monitored throughout development. The probe provides both a Serial Wire Debug (SWD) interface for programming and debugging the Raspberry Pi Pico 2 and a UART interface for serial communication between the microcontroller and the host computer. Through the UART connection, a common GND is established between the two devices, while GPIO 0 on the microcontroller is configured as the transmit pin (TX), and GPIO 1 is configured as the receive pin (RX), allowing the microcontroller to transmit information regarding the current operating state of the synthesizer and receive serial data from the external computer.

2. Software

2.1 General Overview

The software is organized into two concurrent pieces of code. The first is the ISR, which is responsible for the real-time signal synthesis. The ISR (Interrupt Service Routine) works with the protothread (PT_Thread), which holds all the code that manages inputs and control of the various modes included. The ISR is preemptive in the sense that it works off of timer interrupts that get reset after each run-through of the ISR occurs. The PT_Thread is not preemptive, forcing us to yield to allow other threads to run. For our protothread, we are using the “pt_cornell_rp2040_v1_4.h” file that contains the implementation of thread creation and thread manipulation for our scheduler. The scheduler used is a simple round-robin scheduler that allows a thread to complete work and cycles through all the threads in a FIFO (First-In, First-Out) scheme.

The divide between the ISR and PT_Thread keeps timing synthesis away from the much slower event-driven keypad scanning and mode management. We do this so that all the work in the ISR is completed before another interrupt is fired. The ISR is scheduled to fire once every 20 microseconds. We must ensure that the code in the ISR completes much quicker than those 20 microseconds to allow time for the threads to run. This also gives us the constraint of ensuring that the ISR code cannot exceed 20 microseconds in terms of execution time. If this were to

occur, we would get stuck in a cycle of attempting to complete the ISR and then interrupting to start the ISR code once more.

The ISR and PT_Thread communicate with each other through a small set of volatile global variables. These variables must be volatile to force the compiler not to optimize checking these variables. Without volatile, when these variables get updated, the code checking them will see stale values and would have completed the wrong work due to receiving the wrong info.

2.2 Initialization and Frequency Input

The main function performs one-time setup for everything necessary before we can hand off control to the PT_Thread scheduler. The first line in main overlocks the system to 150 MHz before configuring anything else. We need to do this first because many of the subsequent timing-dependent peripherals (ADC, SPI, alarm) use this for their own timing behavior.

Next, we initialize the stdio using “stdio_init_all.” We use this to set up the UART communication, which is used by every printf in the program. We used this to debug the ADC code. It allowed us to check if the value from the ADC was changing when we slid the potentiometer up or down and made sure it was stable when it’s not moving. The early initialization of the stdio is tested with a printf statement holding “Hello, DAC!\n.”

The ADC is first set up using adc_init. We then use adc_gpio_init with ADC_PIN as its input. ADC_PIN holds the value of 26, so we set up the ADC to GPIO26 for analog input. adc_select_input is given the value of ADC_MUX (which holds the value of zero) to select channel 0. We then use this input path every loop iteration in the PT_Thread to produce our curr_freq. We compute this value by taking the value from the ADC and multiplying it by 2.5 so that we have a maximum frequency of about 10 kHz.

We set up GPIO 25 as an output and drive it high. This is so we have an LED blinking to tell us if the main loop is still running. The main loop should never end due to the while(1) loop at the end of main, and this LED is an indicator of that. The next block of code configures the gpio’s used by the keypad. It sets up four rows and three columns used by scan_keypad(). gpio_init_mask brings up all the keypad-related pins in one go. We then set all the column pins as inputs. gpio_set_dir_masked sets the remaining pins as outputs. Those same output pins get set to low. We then finally turn on pull-down resistors for the column pins.

After setting up the keypad gpio's, we initialize the SPI channel and set the baud rate to 20MHz while formatting to 16-bit transfers. We do this to match the DAC's 16 bit format. This is later used by the ISR's call to the function `spi_write16_blocking`. After this, we can map the MISO, SCK, MOSI, and CS to their SPI peripheral function. We use a for loop to produce all 256 entries to our sine table. Scale `sin()` by 2047 so the table spans from -2047 to 2047. We do this once before we start the scheduler so that the ISR only needs to do a lookup when an interrupt occurs.

After sine table setup, using `hw_set_bits` on the timer's interrupt register, the alarm interrupt is enabled. `Alarm_IRQ` is set in the exclusive handler of `alarm_irq`, our ISR. We then enable the interrupt so it can be set off for the first time. Finally, we arm the interrupt to fire after 20 microseconds. With everything set up, we can add our `PT_Thread` to the scheduler. `Pt_schedule_start` is left as the last line of code on purpose. If we start the scheduler any earlier, we'd miss out on any setup code properly running. Once our scheduler is started, we have a `while(1)` loop to make sure we never leave main.

2.3 Direct Digital Synthesis and Interrupt Service Routine

To produce a pure tone, the audio output requires an analog voltage waveform at the frequency specified by the user. Because the Raspberry Pi Pico 2 is a digital microcontroller, it is unable to produce continuous analog voltages. So, instead, we proceed with integrating Direct Digital Synthesis along with an MCP4822 12-Bit DAC to convert the Pico's digital values into an analog voltage waveform.

To have a smooth sine wave, each point along the wave must be generated with equal time intervals to avoid distorting the waveform. During initialization, `main()` was used to configure the on-board hardware timer for 20 μ s, which interrupts the Pico's processor and jumps to the interrupt service routine, `alarm_irq()`. Figure One shows the full interrupt service routine. The ISR serves three main functions: 1. rescheduling the next interrupt for 20 μ s from the last interrupt, 2. deducing the next sine-wave cycle position within the sine table, dependent on the requested frequency, and 3. configuring the sample into a 16-bit command word that the DAC can receive.

```

// Alarm ISR
static void alarm_irq(void)
{
    // Assert a GPIO when we enter the interrupt
    gpio_put(ISR_GPIO, 1);

    // Clear the alarm irq
    hw_clear_bits(&timer_hw->intr, 1u << ALARM_NUM);

    // Reset the alarm register
    timer_hw->alarm[ALARM_NUM] =
        timer_hw->timerawl + DELAY;

    // DDS phase and sine table lookup
    phase_accum_main += phase_incr_main;

    DAC_data =
        (DAC_config_chan_B |
         ((sin_table[phase_accum_main >> 24] + 2048)
          & 0xffff)); // changed to channel B

    // Perform an SPI transaction
    spi_writel6_blocking(SPI_PORT, &DAC_data, 1);

    // De-assert the GPIO when we leave the interrupt
    gpio_put(ISR_GPIO, 0);
}

```

Figure 3: *Interrupt Service Routine used for updating the DDS's Current Position in the sine-wave cycle, selecting sine table value, and constructing the samples for transmission to the DAC.*

Once `alarm_irq()` is executed by the program, GPIO 2 is set high as an indicator that the ISR has successfully begun. We start by clearing the interrupt status bit of alarm 0, then adding $20\mu\text{s}$ to the value in the Pico's timer, and finally sending that time to alarm 0 to schedule the next interrupt. `curr_freq` stores the requested frequency calculated from the potentiometer's ADC, or it can store recorded frequency values. It tells how many sine cycles per second should be produced to produce the desired sound. Since the ISR is executed every $20\mu\text{s}$, with a sampling

frequency $F_s = \frac{1}{T_s} = \frac{1}{20 \times 10^{-6}} = 50,000$ samples per second, and one DDS sample is produced during each execution, the frequency must be expressed in a way that tells the fraction of the sine wave cycle that should be transmitted each sample. To do so, we simply divide the current frequency by the sampling frequency, and scale it by 2^{32} , since DDS uses an unsigned int, `phase_accum_main`, to track the current position within a sine wave:

$$\text{Phase_incr_main} = \frac{\text{curr_freq} \times 2^{32}}{F_s} \frac{\text{cycles/second}}{\text{samples/second}} = \frac{\text{curr_freq} \times 2^{32}}{F_s} \frac{\text{cycles}}{\text{sample}} .$$

This is taken care of separately from the ISR in the protothread since its timing constraints are not as stringent as the $20\mu\text{s}$ hard requirement for the ISR. The protothread calls `PT_YIELD_usec()` after calculating `phase_incr_main` to pause itself and allow the rest of the code to continue. This takes place during the normal operation branch of the protothread:

```
phase_incr_main =
    ((curr_freq * two32) / Fs) * phase_toggle;
PT_YIELD_usec(10000);
```

When `phase_toggle = 0`, the phase increment outputs a constant value, whereas when `phase_toggle = 1`, the phase increment value is retained. After scheduling the next interrupt, the ISR calculates the new phase position by adding the phase increment to the accumulated position within the waveform cycle: `phase_accum_main += phase_incr_main`. During initialization, the `main()` function of `dactest.c` calls `sin()` to fill a table, `sin_table`, to create an array consisting of 256 (or 2^8) positions spaced equally between 0 and about 2π radians. Each index within the array, `sin_table`, holds the value of the height of the sine wave at that position on the sine wave. The 32-bit accumulator represents one complete sine wave cycle as 2^{32} positions, so dividing those positions by the 256 table entries gives that each table entry has a section of the sine wave with 2^{24} possible accumulator positions. To choose the specific index within the table which contains the sine-wave amplitude at the DDS's current position in the cycle, the phase accumulator is shifted 24 bits to the right, which produces an index from 0 to 255: `phase_accum_main >> 24`.

After the ISR determines the proper index and retrieves the amplitude at the DDS's current position in the sine cycle, the values must be received and sent to the digital-to-analog converter. In this lab, we utilized the MCP4822 12-bit DAC, which only accepts unsigned

values from 0 to 4095. The sin table contains signed values from -2047 to +2047. So, if 2048 is added to any given value within the table, it will correspond to a value from 1-4095, which are values that the DAC can accept. *Figure 1* shows the exact line by line as to how we select the sine amplitude, convert it into a range the DAC can accept, and add it to the DAC instructions. The ISR sends the 16-bit commands through the Serial Peripheral Interface (SPI) to the MCP4822, with the last 12 bits containing the shifted sine wave sample. GPIO 2 is then driven low to indicate that the ISR has completed.

2.4 Keypad Scanning and Debouncing

The `scan_keypad` function works by setting a row line high. We then add in a single microsecond delay. This delay is necessary. As we need to consider the switch in the keypad actually moving (this is not instantaneous), we need to give the switches time to move into place when the key is being pressed or let go of. We then read the keycode by using `gpio_get_all`, shift the result by 9, and then mask with `0x7F`. We then check if multiple buttons are pressed. If this is true, we break and return -1. We then check if one button is pressed, and if it is, we compare the keypad with the keycodes of 0-11 or -1 for no valid press. We do this process four times per call of `scan_keypad`, as we have four rows. The idea is to check each key one at a time. With this function, our FSM only ever sees a keycode.

Every Key that we work with has a switch that goes down or up to close or open our circuit, respectively. Because these are physical parts, when they move into place, they bounce. This bounce causes our `scan_keypad` function to record multiple presses even if only one occurs due to the bounces. To combat this, we build a debounce state machine. This FSM holds 4 states:

- **STATE_NOT_PRESSED:** This is the default state and where the FSM stays when idle. Keycodes seen here are not fully trusted, so we move into **STATE_MAYBE_PRESSED** and record the potential press.
- **STATE_MAYBE_PRESSED:** In this state, if the same keycode is read again on the next cycle, we can now trust this key press and act on it. We also use this state for our mode entry logic before entering **STATE_PRESSED**. In this state, if we read a different keycode or no longer read any keycode, we do not act on them and move into the **STATE_NOT_PRESSED** state.

- **STATE_PRESSED:** In this state, we are fully certain that a key is being held. If the `record_flag` is high and the keycode is between 1-9, we store the frequencies being outputted by ADC (the value `adc*2.5` as discussed earlier). We stay in this state until a change in keycode is found. Once a change is found, move to **STATE_MAYBE_NOT_PRESSED**.
- **STATE_MAYBE_NOT_PRESSED:** If the keycode we had used to get into **STATE_PRESSED** is found again here, move back into **STATE_PRESSED**. If it doesn't, we finalize our recording. If the just-released key is equal to a key that contains a recording, we play the previously recorded frequency from that key.

2.5 Operating Mode Logic

Three interactive modes were designed inside the debounce state machine. Each of these modes ultimately augments `curr_freq`, and incidentally `phase_incr_main` is set. All of this logic is handled outside of the ISR so that we don't increase how long we need to remain in the ISR per interrupt. We can toggle the sound on and off by using the key 0. We do this by using the flag `phase_toggle`. Multiplying `phase_toggle` by the calculation for `phase_incr_main` allows us to shift between that value and zero. Effectively turning off the sound. On startup, `phase_toggle` is set to 1 so sound plays when the code first runs. While double-checking a key press in **STATE_MAYBE_PRESSED**, we can set `phase_toggle` to zero (or from zero to one).

Record mode is entered by pressing the key * on the keypad. After a confirmed press of * (which holds the keycode 10), we set the `record_rdy` flag, clear the `playback_flag`, and force `phase_toggle` off so no sound plays until ready. We know we're ready when we press one of the keys ranging from 1-9. Once one of these keys is pressed and the `record_rdy` flag is set, the `record_flg` flag is set. Before we start saving them, we clear the entire row and length. Using the keycode (`keycode - 1`), we can find where to store the frequencies being played.

```
stored_freq[record_key - 1][record_index] = curr_freq;
record_index++;
```

While in **STATE_PRESSED**, we append `store_freq` with the current frequency being played. Each loop through increments `record_index`. We limit this to 1000 samples (`MAX_SAMPLES`), giving 10 seconds of recording. Once the key is released and that is confirmed, the last recorded sample is at `record_key - 1`, and all record flags are cleared. When

pressing a number that has a recording in it (and `record_rdy` is 0), the `playback_flg` is set and a block in the state machine increments the `playback_index`. This is done to replay all of the recorded values by feeding `stored_freq` values into `curr_freq` until the `store_length` is reached. `Phase_incr_main` is zeroed out at the end of a playback to stop the sound.

```
curr_freq = stored_freq[playback_key - 1][playback_index]; // Got
playback_key back in STATE_MAYBE_PRESSED
playback_index++;
```

Compose mode is turned on once a press of # (keycode 11) is confirmed and the `compose_flg` is currently cleared. Set the `compose_flg` and wait for key presses. Any key presses confirmed while in compose mode are appended in order in `compose_keys` with a maximum of `MAX_SAMPLES` (1000) keys stored. Compose length is updated for every confirmed press to track how many keys have been pressed. Pressing # again in this state ends the recording of keys and sets the `compose_pb_flg` while clearing the `compose_flg`. Now we can loop through the compose keys, setting `playback_key` equal to `compose_next_key`.

```
// Compose Update
if ((compose_pb_flag == 1) && (playback_flg == 0) && (compose_pb_index <
compose_length)) // If in compose mode and trying to record, and
playback_flg (normal pb) is LOW
{
    compose_next_key = compose_keys[compose_pb_index];

    // Only play it if that key actually has a recording stored.
    if (recording_length[compose_next_key - 1] > 0)
    {
        playback_key = compose_next_key;
        playback_index = 0;
        playback_flg = 1;
    }
}
```

Figure 3: Composite Mode Playback Logic

Before trying to play anything back, it's important to check if there is something to play, and that is done by checking if the compose index is less than the compose length (as seen in the code above). `playback_key` is set in this manner to reuse the playback code designed earlier. This updates `curr_freq` with all values associated with each key (i.e. all recordings from key 1 and

then key 2). You are allowed to record the same key multiple times in a row this way. Pressing the # key once more resets the playback flags and sets the compose flags, allowing us to record a new set of keys. With this method of compose mode, we do not need to store the frequencies collected using record mode. We only call upon them using the stored key values in `compose_keys`.

2.6 PT_Thread Timing

Our team decided to use only one thread and as a consequence, we needed to add in multiple `PT_YIELD_usec()` 's throughout the thread to control timing. There are three instances in which we yield in our thread, each having to do with `phase_incr_main`.

- Default `PT_YIELD_usec(10000)`: This yields for 10 ms (100Hz) and gives us a tone augmented by the `adc_value`. Fast enough for the human ear (unless you are particularly sensitive to frequencies) to perceive sound without irritation.
- Record `PT_YIELD_usec(10000)`: It is a must to yield for 10 ms so that we record at the same frequency we were listening to by default.
- Playback `PT_YIELD_usec(1000)`: In this scenario, we yield every millisecond (1000Hz), effectively speeding up the playback. This gives a 10-times speed increase from when the frequencies were recorded.

We used `yield` as a means to control how often `curr_freq` is updated and, as a consequence, how fast `phase_incr_main` is updated.

2.7 Testing

The ISR must begin and end within 20 μ s, since it is executed every 20 μ s. Additionally, the full 16-bit command word must be sent to the DAC before that 20 μ s is over. To test that the ISR is completed within that timeframe, we connect GPIO2 to channel 2(blue) of the oscilloscope and the SPI clock signal to channel 1 (yellow). The width of the GPIO pulse is approximately equivalent to the amount of time that it takes the ISR to execute. The oscilloscope shows that the ISR executes for approximately 1.08 μ s, and that the command word is sent to the DAC within that time. Additionally, the frequency of 50 kHz indicated by the oscilloscope confirms the sampling rate is successfully 50,000 samples per second.

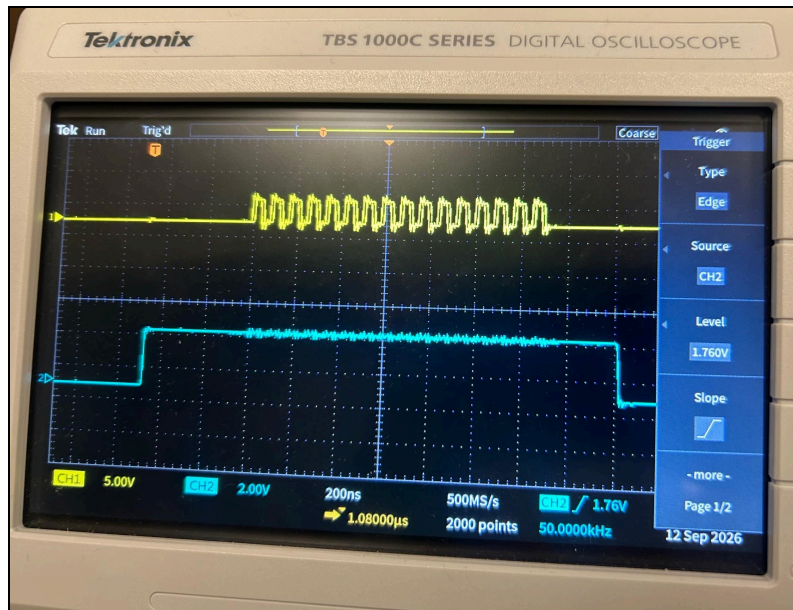


Figure 4: An oscilloscope trace which demonstrates 16 SPI pulses within the 20-microsecond window of the ISR (denoted by the 50kHz indication on the bottom right $1/50\text{kHz} = 20\mu\text{s}$).

The key and debouncing functions are tested by pressing, holding, and releasing each key to ensure that each key is recognized, that one physical press of each key only causes one corresponding action, and that 0 silences the output of the synthesizer, * enters record, and # enters composite mode. During initial testing, printf statements were used to track the keycode and debounce state over UART.

3. General Testing Strategy

A variety of testing methods were implemented in this lab which allowed us to confirm the general functionality of the bird-song synthesizer as it was being built. When testing general circuit correctness and proper connectivity between components (to see if any wires had become misaligned or put into the wrong places on the breadboard), the oscilloscope was used quite often to diagnose missing waveforms (for the ISR (across GPIO2), the system clock, etc.) with the help of additional wires that connected out of the board to the oscilloscope. When testing if our code had any errors we would first see if the code compiled at all. If the code did not compile or run, we turned to the Terminal to read any error messages to see what needed to be edited, such as if we were missing any inclusions from the CMakeFile or improperly declaring an array or variable. Finally, when testing our device functionality, we made sure to first slow the program

down using `PT_YIELD_usec()` of a really large number (>1000000) and use the Debugger Probe to communicate to the Serial Monitor via UART with included `printf` trace statements that allowed us to see where a program was able to get to as well as what the program was outputting. If everything seemed to be working, we would try to implement a stress testing strategy and attempt to overload the program with inputs through various methods of spamming the keypad presses and moving the slide potentiometer back and forth very quickly to see if the circuit would break. We would then unplug the circuit and step back through the traces and `printf` statement outputs to see if the code was properly functioning.

4. AI Use

While AI with the assistance of code generation in this laboratory, it was not used without consideration of the AI Course Policy. All lines of code that were inspired or generated by AI's guidance are properly documented within the code file appended at the end of this report. A prompt log is also provided at the end of this report, which demonstrates that the ways in which AI was used were purely conceptual, asking for help understanding concepts of both the assignment requirements and proper C code documentation (as it had been a while since any of us had implemented C code) rather than directly asking for code to complete the lab checkoffs.

Results

At a high level, the result of this lab was the successful recreation of an audio synthesizer capable of recording, playing back, and composing audio frequency sweeps that a user inputted through adjusting the slide potentiometer. We were unfortunately unable to trick Merlin into believing the repeated swoop-chirp-pause sequence we created was that of a northern cardinal, but we were able to produce a spectrogram plot of frequencies that quite closely resembled that of a northern cardinal's song, as indicated by Figure 5, which demonstrate the spectrogram of a real northern cardinal bird (left) in comparison with our synthesizer (right).

Additionally, Figure 6 demonstrates a spectrogram showing the functionality of Compose Mode, where a swoop-chirp-pause sequence of recordings is played first on Compose Mode and then each element of that sequence is played individually immediately following the Compose Mode sequence, demonstrating Compose Mode's functionality in making a sequence of discretized key presses play back as a continuous output. Figures 7 and 8 further attest to the

overall functionality of our synthesizer, as they present scopes of a bird swoop frequency outputs as oscilloscope traces. Figure 7 demonstrates a zoomed out view of a series of swoops played immediately in succession while Figure 8 shows a closer, more zoomed in view to demonstrate the changing sinusoid being generated based on the varying ADC values.

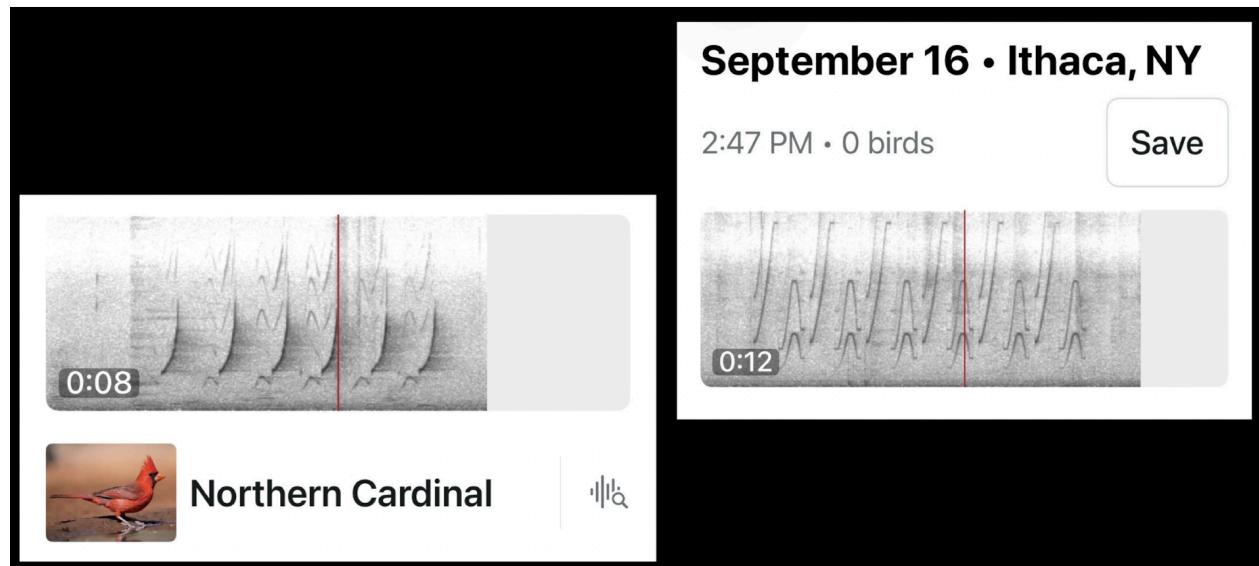


Figure 5: A side by side comparison of spectrogram plots (using Merlin) between the real Northern Cardinal bird and the bird-song synthesizer. While it may look far-off in accuracy, the general shape and frequencies (as indicated by lines on both plots staying above the dark bottom bar and just entering the white cloud at the top) of the swoop-chirp-pause sequence (repeated ~6 times) was similar enough that Merlin entered a “Thinking” state.

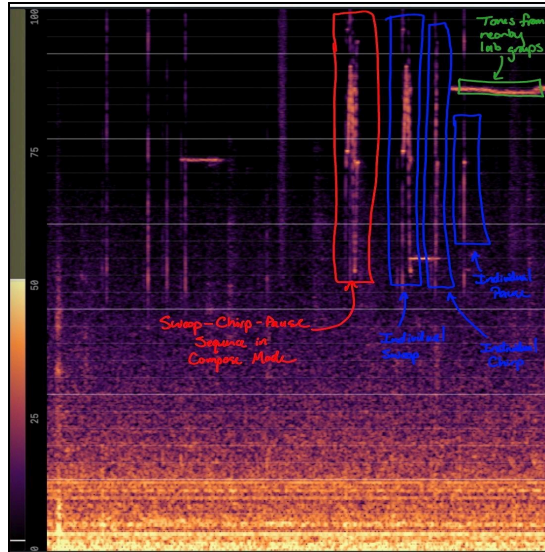


Figure 6: A spectrogram plot demonstrating the functionality of the final version of the synthesizer in “Compose Mode”. Notice how the sum of the three blue plots sums to the red.

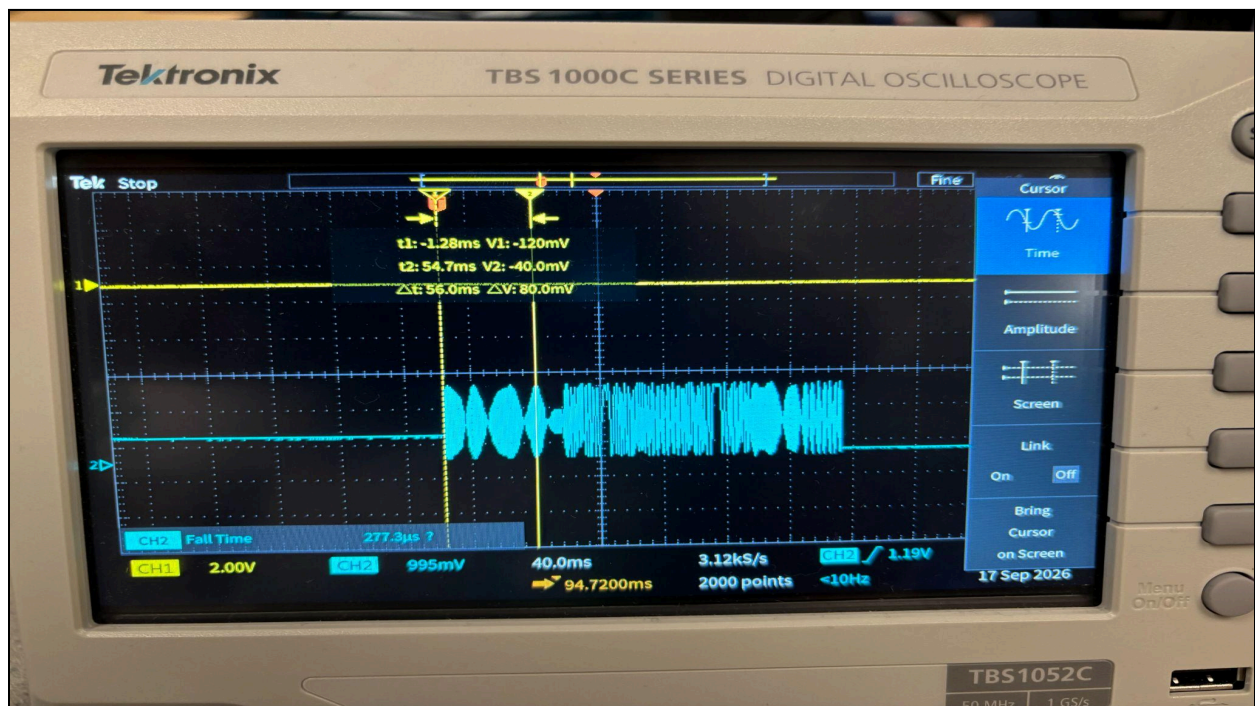


Figure 7: A zoomed out view of multiple (4) identical bird swoop recordings on the oscilloscope trace. The yellow markers roughly outline the expected length of one bird swoop ($\Delta t = 56.0$ ms). Notice the signal looks peculiar due to the fact that the oscilloscope aliases as the length of the song is zoomed out, and thus has lower resolution.

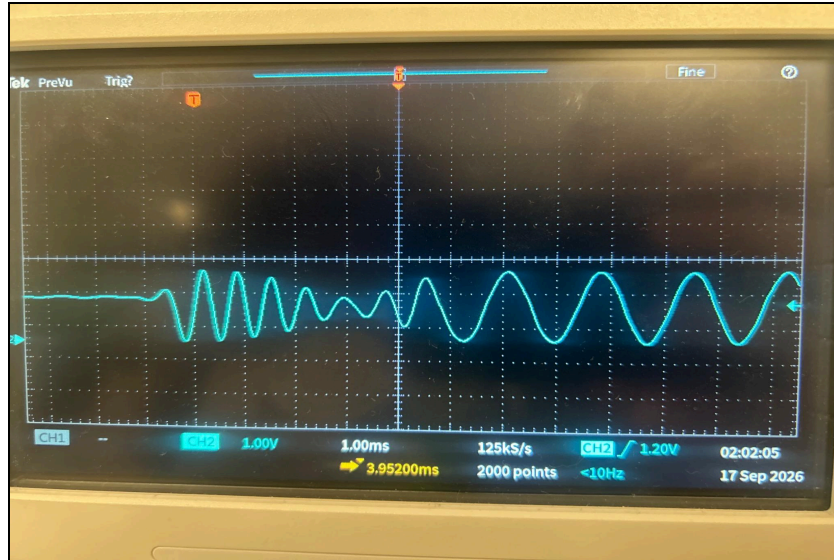


Figure 8: A zoomed-in view of the bird sloop recording on the oscilloscope trace. Notice how the sinusoidal wave is changing in terms of amplitude and how the signal does not alias due to the shortened sloop recording.

With the code appended at the end of this report, we successfully were able to implement each of the lab's features. Upon compiling the code to the microcontroller and running it after putting the board into BOOTSEL mode by shorting pins 30 (RUN) and 28 (GND) together, the synthesizer begins producing a constant tone which changes in real time as a user adjusts the positioning of the slide potentiometer on the board, and thus how much current is allowed to flow, causing different frequencies to be emitted. Pressing the "0" key mutes the tone generator, and pressing the "0" key again will unmute the tone generator, once again allowing the frequencies to be heard.

Pressing the star key ("*") and then pressing and holding any key marked between "1" and "9" causes the synthesizer to enter "Record Mode", meaning that so long as the user continues to hold down a key between "1" and "9", a constant connection pulled down between the input column GPIOs to the output row GPIOs will be kept (depending on the button); the synthesizer will store the frequencies changed by the adjustment of the slide potentiometer and outputted by the DAC into a 9x10000 array indexed every millisecond. The rows of the array are indexed based on which key is pressed, and each row is able to hold 10,000 frequency samples, therefore allowing 10 seconds worth of recording per key before the program exits "Record

Mode”. Pressing the number key that was recorded on again will then cause the program to enter “Playback Mode”, where the stored frequencies in the array will be played back out on the DAC.

Pressing the pound key (“#”) causes the program to enter “Compose Mode”, meaning that any sequence of key presses (with pre-recorded frequency sweeps on them from “Record Mode”) that the user inputs on the keypad will be stored in a 1x10000 array. Upon pressing the “#” key again, the synthesizer will enter “Playback Mode” and go through the array, playing the frequency sweeps associated with each of the keys pressed but 10x faster such that the chirps outputted by the DAC are sharper and more shrill. Once the synthesizer is finished playing back all the frequencies of the associated keys in sequence, if the “#” key is again pressed, the array containing the sequence of keys pressed will be cleared such that the next composition of keys can be recorded.

Below are three separate finite state machines which demonstrate the logic implemented in the code’s protothread. In the code, each of “Record Mode” (Figure 9), “Playback Mode” (Figure 10), and “Compose Mode” (Figure 11) exists under the same 4 states across cycles, but for easier viewing purposes in this report, each was separated into its own relevant path. State names will be abbreviated in the captions of the figures for simplicity (e.g. STATE_MAYBE_PRESSED = STATE_MP, STATE_NOT_PRESSED = STATE_NP, etc).

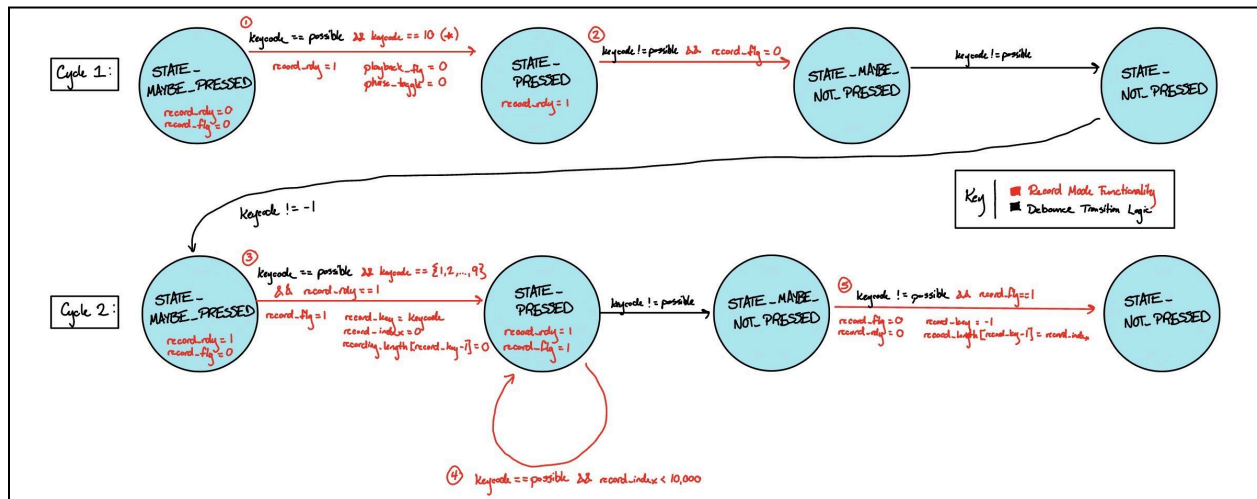


Figure 9: “Record Mode” FSM. In Cycle 1, if the star key (keycode = 10) is selected in STATE_MP, set the record_rdy flag high, indicating that on the next cycle the program knows that a previous instance of keycode = 10 has been recorded, and any press and hold of a number key will enter into “Record Mode” for that key. On Cycle 2, with the record_rdy flag high and

continual pressing of a positive number key (1-9), set the `record_flg` high in `STATE_MP` so that upon entering `STATE_P`, so long as the key is held down, the current frequency is indexed from the slide potentiometer to the number key's specific row within the 9x1000 array and the length of the recording is tracked. Exiting `STATE_P` and then on the next cycle `STATE_MNP` resets the flags low.

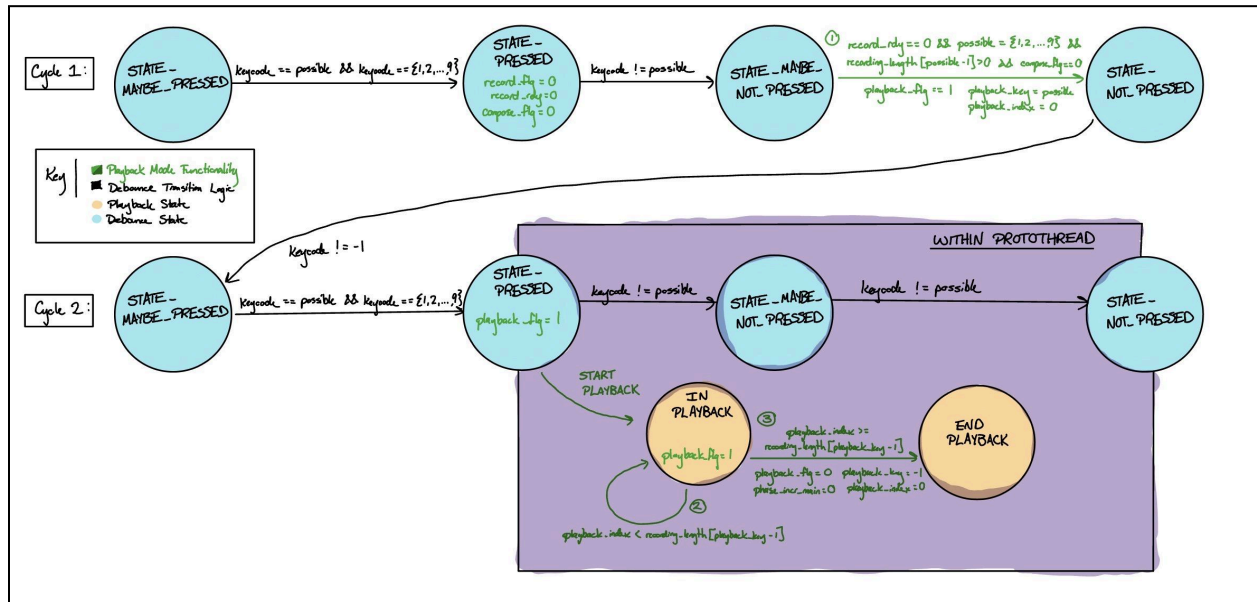


Figure 10: “Playback Mode” FSM. In Cycle 1, if a positive number key (1-9) is pressed but the program has no prior recollection of the `record_rdy` or `compose_flg` flags being set high, then the program knows it is trying to playback. On Cycle 2, so long as the recording length for the key pressed in `STATE_MNP` is greater than 0, we set a `playback_flg` high so that when we press the key we can exit the FSM logic and enter into the thread, where so long as the `playback_index` is less than the `recording_length` of the key pressed, the synthesizer will playback the index stored in the 9x1000 frequency array. Once the thread has indexed through the whole recording, the thread will set the flags low again.

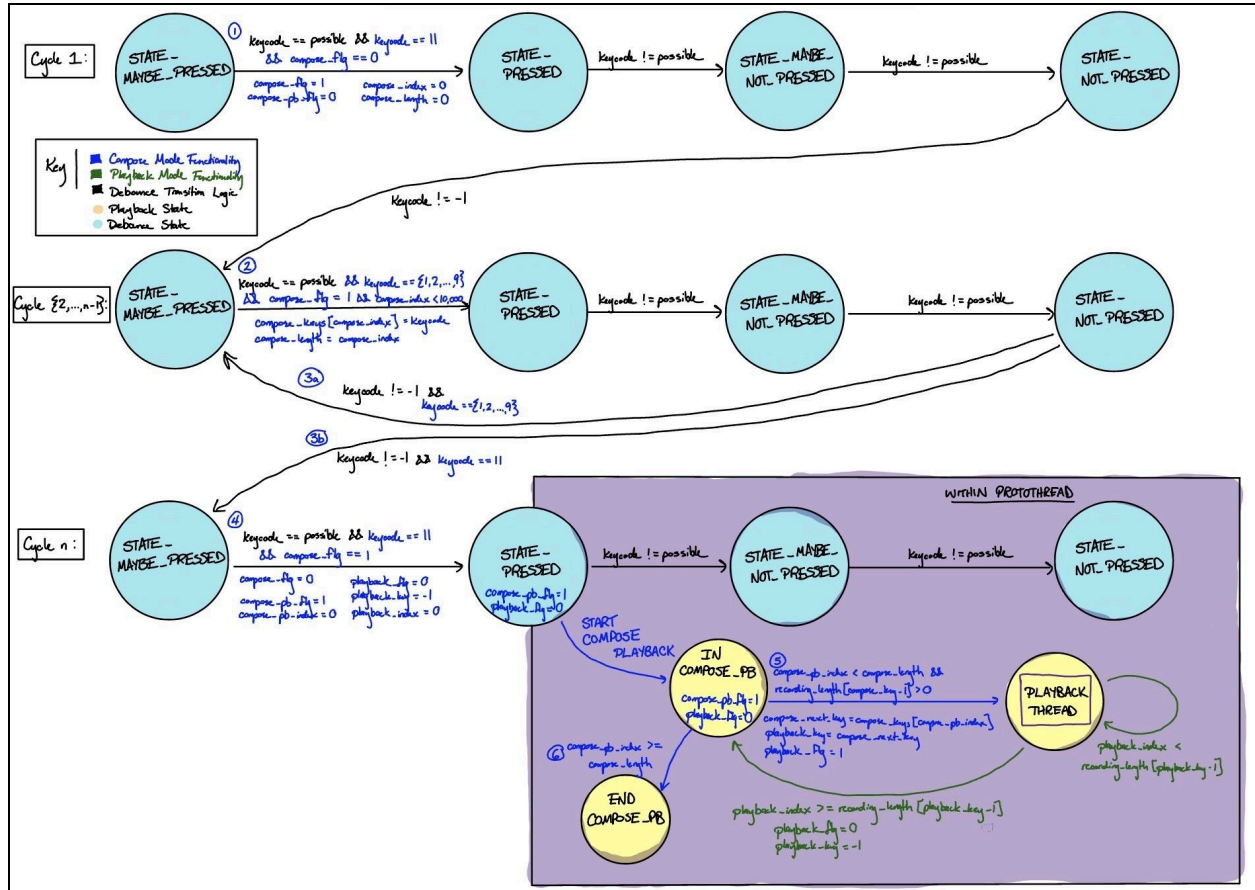


Figure 11: “Compose Mode” FSM. In Cycle 1, if the pound key (keycode = 11) is pressed in STATE_MP but the compose_flg is low, the program sets the compose_flg high meaning that if a number key (1-9) is pressed on the next cycle, now that the compose_flg is high the program knows to enter the recording part of Compose Mode and record the keys pressed. On the next n cycles, so long as the user inputs number keys (1-9) the FSM logic will loop, each time indexing a 1x1000 array with the keycode of the key pressed. When all the desired keys presses have been recorded, if the pound key is hit again (keycode = 11), the program will see that the compose_flg is high and subsequently set it low and compose_pb_flg high to indicate that the program is entering the playback part of compose mode. On the Cycle n+1, then, with the compose_pb_flg set high, the program exits the FSM logic and enters the thread updates for compose and playback modes. So long as there is a keycode stored in the 1x1000 array that has not already been played back, the program will enter “Playback Mode” with the playback_key set as the compose_next_key and the playback_flg high. For that key, the program will step through the recorded frequencies in the 9x1000 array for that key and play them back until the recording is complete. Then it will circle back, setting the playback_flg low again to re-enter “Compose

Mode” update and see if there is another key that is in the 1x1000 that needs to be played back. The program then re-enters “Playback Mode” after setting the playback_flg high again with this new key and steps through its recording until it has finished recording. This process repeats until there is no new key in the 1x1000 array to play back.

Timing and Playback Speed

The Interrupt Service Routine is executed every 20μs with a sampling frequency of $F_s = \frac{1}{20 \times 10^{-6}} = 50,000$ samples per second. *Figure 4* shows the oscilloscope testing to showing how long the ISR executes. Though the ISR’s specific time constraint is a runtime of less than 20μs, only 5.4% of the ISR period is used during an interrupt:

ISR Period Usage = $\frac{1.08 \mu s}{20 \mu s} \times 100 = 5.4$, with $20 \mu s - 10.8 \mu s = 18.92 \mu s$ before the next interrupt is executed.

The’s CPU Pico operates at 150 MHz, or 150,000,000 cycles per second. Since the ISR operates with an interrupt period of, 20μs, there are

$150,000,000 \text{ cycles per second} \times 20 \times 10^{-6} \text{ seconds} = 3000$ cycles available to be occupied by the ISR. The ISR occupies

$150,000,000 \text{ cycles per second} \times 1.08 \times 10^{-6} \text{ seconds} = 162$ cycles. So, it uses about 5.4% of the processor's available cycles ($\frac{162}{3000} \times 100 = 5.4$), with 2838 cycles remaining.

The playback speed of the recording is directly tied to the value stored in curr_freq. During playback, curr_freq is updated every millisecond, and during recording or normal operation using the potentiometer, curr_freq is updated every 10μs. This means playback moves through the same sound 10 times faster without changing the frequency stored in curr_freq. Each key on the keypad is able to store up to 1,000 samples. That means the maximum recording time per key is $1000 \text{ samples} \times 10 \frac{\mu s}{\text{sample}} = 10 \text{ seconds}$. If a recording contains N samples, its original duration is $N \times 10 \mu s$, and its playback duration is $N \times 10 \mu s$.

Error Analysis

The 12-bit analog-to-digital converter allows values from 0-4095. To calculate curr_freq, we multiply the ADC value by 2.5. So, as ADC increments by 1 from zero to 4095, it increases

by an average frequency of 2.5 Hz. This gives a frequency range of zero through $4095 \times 2.5 = 10,237.5 \text{ Hz}$ which can be stored in curr_freq. Since curr_freq is an int, all values within the decimal position are disregarded. Table 3 shows the difference between the calculated ADC values for the range 0-10 and their truncated counterpart stored in curr_freq.

ADC value (adc_val)	Frequency calculation (adc_val $\times$ 2.5)	Stored curr_freq
0	$0 \times 2.5 = 0.0 \text{ Hz}$	0 Hz
1	$1 \times 2.5 = 2.5 \text{ Hz}$	2 Hz
2	$2 \times 2.5 = 5.0 \text{ Hz}$	5 Hz
3	$3 \times 2.5 = 7.5 \text{ Hz}$	7 Hz
4	$4 \times 2.5 = 10.0 \text{ Hz}$	10 Hz
5	$5 \times 2.5 = 12.5 \text{ Hz}$	12 Hz
6	$6 \times 2.5 = 15.0 \text{ Hz}$	15 Hz
7	$7 \times 2.5 = 17.5 \text{ Hz}$	17 Hz
8	$8 \times 2.5 = 20.0 \text{ Hz}$	20 Hz
9	$9 \times 2.5 = 22.5 \text{ Hz}$	22 Hz
10	$10 \times 2.5 = 25.0 \text{ Hz}$	25 Hz

Table 3: ADC readings from 0 through 10, the requested frequency, and stored curr_freq

Odd ADC values produce a decimal of 0.5, which is lost during truncation, introducing about 0.5 Hz of error:

Truncation Error: $-0.5 \text{ Hz} \leq \text{error} \leq 0 \text{ Hz}$.

Since the cardinal frequency range is 2 -7 kHz, the maximum error (occurring at 2kHz)

$$\frac{0.5}{2000} \times 100 = 0.025\%.$$

Most gaps between the calculated frequency and the value stored in curr_freq are between 2 and 3 Hz. Additionally, the DDS phase increment is also stored as an int value, specifically an unsigned int called phase_incr_main. Each produced decimal value is also

disregarded. Each phase increment is $\frac{F_s}{2^{32}} = \frac{50000}{4294967269} \approx 0.00001164 \text{ Hz}$. This represents the *DDS Error*: $-0.00001164 \text{ Hz} \leq \text{error} \leq 0 \text{ Hz}$. The maximum error (also occurring at 2kHz) is thus $\frac{0.00001164}{2000} \times 100 = 0.000000582\%$. These small inconsistencies in continuous frequency sweeps are so small that they are indistinguishable to most human hearing, and can trick humans and birds alike into interpreting the sound as a legitimate bird sound.

Conclusion

All in all, this lab was quite useful in providing a practical introduction to a few previously-forgotten fundamental principles of embedded systems, including direct digital synthesis, timer interrupts, analog-to-digital conversion, and serial communication, which we were able to explore while developing a functional digital bird-song synthesizer capable of generating, recording, playing back, and composing unique frequency sweeps that could be used to emulate the swoops and chirps of northern cardinals and other birds. While we ultimately were unable to reproduce a version of the northern cardinal bird's song on the synthesizer that could trick Merlin, our group came tremendously close (as indicated by Figure 5) and we all gained a new appreciation for the applications of digital signal processing, despite none of us being particularly interested in the field prior to this lab.

While this lab frames the overall usability of the device to be a digital sound synthesizer capable of emulating bird songs, with some modifications there are both many similar and other alternative applications that the underlying foundation of the device would be applicable to. For example, if relatively few modifications were made to the underlying synthesizer circuit, the device could very likely be used in the emulation of other frequency sounds both inside and outside the range of human hearing (e.g. cricket chirps, cicada chirps, musical instrument, distress signal, a dog-whistle, etc.), whereas if the underlying foundation was the only thing that remained consistent, then the synthesizer foundation could be applied to alternative applications such as generating multiple sinusoidal waveforms to produce images on an oscilloscope or providing controlled audio-frequency signals as inputs to a polygraph or Richter scale machine.

Despite our awe in seeing the synthesizer successfully demonstrating the desired functionality with its various record, playback, and compose modes, there are several aspects of the assignment that we think could be improved for future lab sections. In particular, one of the

primary challenges we encountered was the limited accessibility of the laboratory equipment outside our assigned lab section (especially after the audio-jack connection to a speaker was introduced), which made it difficult to work ahead and complete the more comprehensive Week 2 checkpoint within the allotted time. Regarding the Week 2 checkpoint, there was simply way too much work to do and the scope of the checkoff was entirely too comprehensive to get done in one week (though it seems the professor is already aware of this issue as he directly asked our group about the feasibility of the implementation scope). Although students are encouraged to attend additional lab sections, scheduling conflicts made this option impractical in some cases, especially given that some of our group members are enrolled in over 18 credits of classes and the sheer amount of time testing and debugging required to implement the recording and playback functionality properly.

Perhaps the most important advice we could give regarding this lab has to do with the uncomfortable environment we had to work in. Having to work in a lab where lab groups are constantly producing annoying, high-pitched sounds trying to get their synthesizers operational was not conducive to a productive work session in labs. Because of the loud pitched noises and constant ear-ringing it was often difficult to hear ourselves think, and many of our group members often developed migraines from the constant noises and had to step out of the lab for a minute to get a break from the noises. Put simply, the lab environment was beyond overstimulating, especially for those in our group with audio-sensitivity issues and SDS accommodations. As a suggestion for a future improvement, we would recommend dividing the lab in half. Groups who want to test audio detection and functionality should go to one side of the lab space while those trying to think and write code should stay on the other side. Additionally, groups actively testing should not be allowed to raise the volume of their speakers beyond 25%. If these suggestions were implemented into lab spaces, we think there would be less occurrences of migraines and general discomfort, and better utilization of lab time for productive work sessions.

AI Usage Discussion

Below is a summary table of all AI usage for code generation in this lab. It includes the number of additions/deletions/modifications suggested and accepted itemized by date as well as the general concepts. The prompt log is listed under the table.

Date	User Prompt / Request	AI Response / Suggested Changes	Suggested Changes (A/D/M)	Accepted Changes (A/D/M)
Sept. 3–4	Requested assistance implementing a keypad finite state machine and understanding the transition logic between button presses and releases.	Explained the four-state keypad FSM and suggested modifications to the state-transition logic to distinguish valid button presses, releases, and mechanical bouncing.	1 / 0 / 1	1 / 0 / 1
Sept. 4–5	Requested assistance implementing Record Mode, allowing the user to press *, hold a numeric key, and store frequency values while the key remained pressed.	Suggested implementing a two-dimensional frequency storage array, recording flags, key identifiers, and logic to store potentiometer readings while a numeric key was held.	4 / 0 / 2	4 / 0 / 2
Sept. 5–8	Requested assistance debugging recording and playback functionality, particularly determining where playback logic should be placed within the existing program.	Suggested separating recording and playback operations from the 50 kHz ISR, moving frequency storage and retrieval into the main protothread, and modifying the frequency update logic.	2 / 1 / 2	2 / 1 / 2
Sept. 8–9	Requested incremental guidance on implementing playback, including initializing playback variables, retrieving stored frequency values, and stopping playback after the final sample.	Suggested adding playback flags and indices, checking the recording length before retrieving stored values, and resetting the playback variables and phase increment once playback was complete.	2 / 0 / 3	2 / 0 / 3
Sept. 9–10	Requested assistance increasing playback speed while maintaining the original recording rate.	Suggested modifying the protothread yield intervals so that stored frequency values could be retrieved more rapidly than they were originally recorded.	0 / 0 / 2	0 / 0 / 2

Sept. 10–2 2	Requested assistance implementing Compose Mode, allowing the user to record a sequence of numeric key presses and play back the corresponding previously recorded frequency sequences.	Suggested implementing an array of key identifiers, composition flags and indices, and conditional logic to retrieve and play individual recordings sequentially.	3 / 0 / 3	3 / 0 / 3
Sept. 21–2 2	Requested assistance reviewing the final ISR implementation and verifying the relationship between the interrupt frequency and the SPI waveform transmission.	Explained the existing DDS phase accumulation, sine table lookup, and SPI transmission operations. Assisted with interpreting oscilloscope measurements of the ISR timing. No additional code changes were required.	0 / 0 / 0	0 / 0 / 0
Total			12 / 1 / 13	12 / 1 / 13

Below is the prompt log and response chains in conversations I had with an AI tool. Within the code itself, all instances of code labeled “AI-GENERATED” was supplied by ChatGPT and incorporated into the program, while code labeled “AI-ASSISTED” was written or revised by us using ChatGPT’s conceptual explanations and debugging guidance but never direct verbatim solutions. Given ChatGPT was mainly used as an inspiration for what to do, as it had been some time since any one of us had written C code, “AI-ASSISTED” tags were applied liberally throughout the code.

Exchange 1: Implementing the Keypad Finite State Machine

User: "How would I implement a Moore FSM in C for the keypad? I want it to recognize when a button is pressed and released, but I also need to account for bouncing."

ChatGPT: Explained how a four-state FSM could distinguish between no press, a potential press, a confirmed press, and a potential release. Suggested using an enumerated type to represent the states and a `switch` statement to control transitions.

User: "So would I put the state variable outside the while loop? And how would I keep track of which key was pressed?"

ChatGPT: Suggested declaring a persistent state variable and storing the detected key in a separate variable so that the keypad could be checked across successive iterations of the protothread.

User: "Okay, I implemented it. Can you check whether my transitions between states make sense?"

ChatGPT: Reviewed the FSM structure and explained how the `possible` variable could be used to confirm whether the detected key remained consistent between successive keypad scans.

Resulting implementation:

```
typedef enum
{
    STATE_NOT_PRESSED,
    STATE_MAYBE_PRESSED,
    STATE_PRESSED,
    STATE_MAYBE_NOT_PRESSED
} state_keypad;
```

Code changes: One addition (FSM definition) and one modification (keypad input-processing logic).

Exchange 2: Developing the Keypad Scanning Function

User: "I want to separate the actual keypad scanning from the FSM so that I can just call a function and use the returned keycode. Is that possible?"

ChatGPT: Explained how a separate function could sequentially scan the keypad rows, read the column inputs, and compare the resulting value against an array of valid keycodes.

User: "Would I make the function void since there are no input parameters?"

ChatGPT: Clarified that a function can have no input parameters while still returning a value. Suggested using an integer return type so that the function could return a valid key identifier or **-1** when no button was detected.

User: "Okay, so I can just set keycode equal to the function and use that in the state machine?"

ChatGPT: Confirmed that the returned key identifier could be assigned to **keycode** and subsequently used to determine the appropriate FSM transitions.

Resulting implementation:

```
int scan_keypad(void)
{
    // Scan the keypad rows and read the column inputs.

    // Return the detected key identifier,
    // or -1 if no valid key is detected.
}
```

The completed function scans the four keypad rows and compares the resulting input against the twelve valid keycodes.

Code changes: One addition (keypad scanning function).

Exchange 3: Creating the Frequency Recording Array

User: "We are trying to store frequency values parsed 10 ms apart in an array in C. How do we denote, once a condition is met, going into the n-1th row of condition = n and storing the associated frequencies?"

ChatGPT: Suggested using a two-dimensional array in which each row corresponds to one of the nine numeric keys and each column stores a frequency sample.

User: "So if the user presses key 3, would that mean we store the values in row 2?"

ChatGPT: Explained that because C arrays use zero-based indexing, key 3 corresponds to row 2, allowing the program to use `record_key - 1` to select the appropriate storage location.

User: "And then I would increment the index every time I record a new frequency?"

ChatGPT: Explained how the recording index could be incremented after storing each frequency and checked against the maximum array length to prevent writing beyond the allocated memory.

Resulting implementation:

```
#define MAX_SAMPLES 1000

volatile int stored_freq[9][MAX_SAMPLES] = {0};
stored_freq[record_key - 1][record_index] = curr_freq;
record_index++;
```

Code changes: One addition (two-dimensional frequency storage array) and one modification (recording index logic).

Exchange 4: Implementing Record Mode

User: "How would I modify the program so that pressing and releasing the asterisk button puts the device into Record Mode? The user should then be able to press and hold any key from 1–9, and the frequencies should be recorded while they hold that button."

ChatGPT: Suggested using separate variables to distinguish between the system being ready to record and actively recording. Explained that pressing the asterisk key could set a readiness flag, while subsequently pressing a numeric key could initialize the recording process.

User: "I don't want to directly copy the code from you. Could we go through this incrementally? It's a little overwhelming."

ChatGPT: Broke the implementation into smaller changes, beginning with the recording flags and key identifier, followed by initializing the recording index and storing frequency values while the selected button remained pressed.

User: "So I should store the frequency in STATE_PRESSED because that's where the button is being held?"

ChatGPT: Explained that the confirmed pressed state could be used to repeatedly store frequency samples while the numeric key remained held, with the recording operation ending once the FSM confirmed the button had been released.

Resulting implementation:

```
static int record_rdy = 0;
static int record_flg = 0;
static int record_key = -1;
static int record_index = 0;
```

The recording logic was incorporated into the existing keypad FSM.

Code changes: One addition (recording variables) and one modification (FSM recording logic).

Exchange 5: Recording Length and Button Release

User: "How do I know when to stop recording and how many frequency samples were actually stored?"

ChatGPT: Suggested maintaining an array containing the recording length associated with each numeric key. The final recording index could be saved when the FSM confirmed that the user had released the button.

User: "Would I put that in STATE_MAYBE_NOT_PRESSED, after confirming the button has actually been released?"

ChatGPT: Explained that the recording length should be updated once the release was confirmed, rather than immediately upon the first potentially unstable keypad reading.

User: "Okay, so then I can use that length later to know when playback should stop?"

ChatGPT: Confirmed that the stored recording length could be used as the upper bound when retrieving frequency values during playback.

Resulting implementation:

```
volatile int recording_length[9] = {0};  
recording_length[record_key - 1] = record_index;
```

```
record_flg = 0;  
record_rdy = 0;  
record_key = -1;
```

Code changes: One addition (recording-length array) and one modification (button-release handling).

Exchange 6: Initializing Playback Mode

User: "I'm still a little unsure how to fix the playback logic and where to put it. Can we go through this incrementally?"

ChatGPT: Suggested introducing separate playback variables to identify the selected recording, track the current playback position, and indicate when playback was active.

User: "Would I put the playback logic inside the FSM or outside of the switch cases?"

ChatGPT: Explained that the FSM could be responsible for detecting the button event that initiates playback, while the actual retrieval of stored frequency values could occur outside the FSM in the main protothread.

User: "So pressing a key sets the playback flag, and then the program checks that flag outside the FSM?"

ChatGPT: Confirmed that this arrangement would allow the program to continue scanning the keypad while updating the playback frequency during successive protothread iterations.

Resulting implementation:

```
static int playback_flg = 0;
static int playback_key = -1;
static int playback_index = 0;
```

Code changes: One addition (playback control variables) and one modification (FSM playback initialization).

Exchange 7: Retrieving Stored Frequencies During Playback

User: "Once playback is enabled, how do I get the program to go through the stored frequencies one at a time?"

ChatGPT: Suggested using the playback key to select the appropriate row of the frequency array and the playback index to retrieve successive samples.

User: "Would it make sense to check whether playback_index is less than recording_length before accessing the array?"

ChatGPT: Explained that checking the recorded length prevents the program from retrieving values beyond the end of the selected recording.

User: "Okay, so after retrieving the frequency, I increment the index and then update the phase increment?"

ChatGPT: Explained that the retrieved frequency could be assigned to `curr_freq` and subsequently used to calculate the DDS phase increment.

Resulting implementation:

```
if ((playback_flg == 1) &&
    (playback_index < recording_length[playback_key - 1]))
{
    curr_freq = stored_freq[playback_key - 1][playback_index];
    playback_index++;
}
```

Code changes: One addition (frequency retrieval logic) and one modification (DDS frequency update).

Exchange 8: Stopping Playback After the Final Sample

User: "How do I make sure playback actually stops after all the frequencies have been played? Right now, I'm worried the program might keep playing the last frequency."

ChatGPT: Suggested checking whether the playback index had reached the stored recording length and resetting the playback variables when the recording was complete.

User: "Would resetting `phase_incr_main` to zero stop the tone?"

ChatGPT: Explained that setting the phase increment to zero prevents the phase accumulator from advancing, causing the DAC to output a constant voltage rather than a changing sinusoidal waveform.

User: "Okay, so I should reset the playback flag, key, and index as well?"

ChatGPT: Confirmed that resetting these variables would return the playback controller to its inactive state and prepare it for another recording.

Resulting implementation:

```
playback_flg = 0;
playback_key = -1;
playback_index = 0;

phase_incr_main = 0;
```

Code changes: One modification (playback termination logic).

Exchange 9: Increasing Playback Speed

User: "I want playback to be about 8–10 times faster than the recording. Is there a way to do this using a rate variable instead of changing the entire playback structure?"

ChatGPT: Explained that the recording and playback rates could be controlled through their respective protothread yield intervals without changing the 50 kHz DAC sampling rate.

User: "So if I record every 10 ms but play back every 1 ms, that should give me approximately 10 times faster playback?"

ChatGPT: Confirmed that the playback frequency sequence would advance at approximately ten times the recording rate, while the DDS ISR would continue generating waveform samples at the same nominal 50 kHz rate.

User: "Would this affect the actual frequency of the sine wave?"

ChatGPT: Explained that the playback interval determines how quickly the program advances through the recorded frequency sequence. The instantaneous waveform frequency remains determined by the current DDS phase increment.

Resulting implementation:

```
// Recording
PT_YIELD_usec(10000);

// Playback
PT_YIELD_usec(1000);
```

Code changes: One modification (playback timing).

Exchange 10: Creating the Compose Mode Array

User: "For Compose Mode, could I store the sequence of keys the user presses instead of recording all of the frequency values again?"

ChatGPT: Explained that storing the numeric key identifiers would allow the program to reuse previously recorded frequency sequences without creating another large array of frequency samples.

User: "So I would just need a one-dimensional array that stores which keys were pressed?"

ChatGPT: Suggested using an array of key identifiers, along with an index and length variable to track the sequence entered by the user.

User: "Would I add the key to the array once the press is confirmed in STATE_MAYBE_PRESSED?"

ChatGPT: Explained that storing the key upon confirmation of a valid button press would prevent the same key from being added repeatedly while the user continued holding the button.

Resulting implementation:

```
volatile int compose_keys[MAX_SAMPLES] = {0};  
compose_keys[compose_index] = keycode;  
compose_index++;
```

```
compose_length = compose_index;
```

Code changes: One addition (composition array) and one modification (keypad FSM composition logic).

Exchange 11: Entering and Exiting Compose Mode

User: "I want the first press of the pound key to enter Compose Mode and the second press to stop recording the key sequence and start playback. How should I handle that?"

ChatGPT: Suggested using a composition flag to indicate whether the program was currently recording key identifiers and a separate flag to indicate that composition playback should begin.

User: "So when I press the pound key the second time, I should set compose_flg to zero and compose_pb_flag to one?"

ChatGPT: Explained that this would distinguish between recording the composition and playing it back.

User: "And I should reset compose_pb_index to zero so playback starts from the first recorded key?"

ChatGPT: Confirmed that resetting the composition playback index would allow the program to begin processing the stored sequence from its first entry.

Resulting implementation:

```
compose_pb_flag = 1;  
compose_flg = 0;  
compose_pb_index = 0;
```

Code changes: One addition (composition control variables) and one modification (Compose Mode transition logic).

Exchange 12: Playing Individual Recordings During Compose Mode

User: "Would I put the Compose Mode playback logic outside the FSM but before the normal playback update code?"

ChatGPT: Explained how Compose Mode could select the next key identifier from the composition array and initialize the existing playback functionality for the corresponding frequency recording.

User: "I don't want to create a separate playback function for Compose Mode. Can I reuse the playback logic that I already have?"

ChatGPT: Suggested allowing Compose Mode to initialize the normal playback variables, so that individual frequency recordings could be played using the same code responsible for ordinary playback.

User: "So I would set playback_key to compose_next_key, reset playback_index to zero, and set playback_flg to one?"

ChatGPT: Confirmed that this would initiate playback of the selected recording. Once the recording was complete, the composition controller could select the next key in the sequence.

Resulting implementation:

```
if (recording_length[compose_next_key - 1] > 0)
```

```
{  
    playback_key = compose_next_key;  
    playback_index = 0;  
    playback_flg = 1;  
}
```

Code changes: One addition (composition playback logic) and one modification (integration with existing playback functionality).

Exchange 13: Completing the Composition Playback Sequence

User: "So essentially what the program does is enter Compose Mode, grab the key, then enter playback mode, and once the recording is done for that key, it re-enters Compose Mode outside of the FSM, grabs the next key, and repeats until all keys have been played?"

ChatGPT: Confirmed that the composition controller uses the existing playback functionality to play each stored recording sequentially. After the selected recording finishes, the playback flag is cleared, allowing the composition controller to advance to the next key identifier.

User: "Does it enter back into STATE_MAYBE_PRESSED with playback_key = compose_next_key?"

ChatGPT: Explained that composition playback does not require the keypad FSM to transition back into **STATE_MAYBE_PRESSED**. Instead, the program directly initializes the playback variables outside the FSM.

User: "Ah, I see. So it enters composition mode, then selects the first index and goes into playback mode outside the FSM and within the thread because playback_flg == 1 now. Then it plays that key until it runs out, checks the compose_pb_index < compose_length condition again, and grabs the next key if there is one?"

ChatGPT: Confirmed that this describes the overall composition playback sequence. The composition controller continues selecting stored key identifiers until the composition playback index reaches the recorded sequence length.

User: "Does the final halt occur upon the final frequency being played back, or does it enter the compose playback mode again and then exit once it sees the condition isn't true anymore?"

ChatGPT: Explained that the normal playback logic first completes the final frequency recording and clears its playback flag. The composition controller then checks whether additional keys remain in the sequence. If the composition playback index has reached the composition length, no additional recording is initiated.

Resulting implementation:

```
if((compose_pb_flag == 1) &&
    (playback_flg == 0) &&
    (compose_pb_index < compose_length))
{
    compose_next_key = compose_keys[compose_pb_index];

    if (recording_length[compose_next_key - 1] > 0)
    {
        playback_key = compose_next_key;
        playback_index = 0;
        playback_flg = 1;
    }

    compose_pb_index++;
}
```

Code changes: One modification (composition playback termination and sequencing logic).

Lab 1 Code:

```
#include <stdio.h>
#include <math.h>
#include "pico/stdlib.h"
#include "hardware/timer.h"
#include "hardware/irq.h"
#include "hardware/spi.h"

// added from ADC_demo
#include "hardware/gpio.h"
#include "hardware/adc.h"
#include <string.h>
#include "stdlib.h"

// added from keypad
#include "pico/multicore.h"
#include "hardware/pio.h"
#include "hardware/dma.h"
#include "hardware/sync.h"
#include "hardware/clocks.h"

// =====
// === protothreads globals
// =====
// protothreads header
#include "pt_cornell_rp2040_v1_4.h"

#define LED_PIN 25
#define ADC_PIN 26
#define ADC_MUX 0

// Low-level alarm infrastructure we'll be using
#define ALARM_NUM 0
#define ALARM_IRQ timer_hardware_alarm_get_irq_num(timer_hw, ALARM_NUM)

// DDS parameters
#define two32 4294967296.0 // 2^32
#define Fs 50000           // WAS 50000 BEFORE
#define DELAY 20           // 1/Fs (in microseconds)
```

```

// ADDED: Max number of samples a recording can hold in the array
// ##### BEGIN AI-ASSISTED CODE #####
#define MAX_SAMPLES 1000 // 100 Hz * .01 s (1 ms) = 100 samples per second, so for max
10 seconds = 1000 samples

// ##### END AI-ASSISTED CODE #####
// the DDS units:
volatile unsigned int phase_accum_main;
volatile unsigned int phase_incr_main = (800.0 * two32) / Fs;

// ADDED: FREQUENCY RECORDING AND PLAYBACK PARAMS
// ##### BEGIN AI-ASSISTED CODE #####
volatile int curr_freq = 0; // Current frequency being played back
or recorded
volatile int stored_freq[9][MAX_SAMPLES] = {0}; // Store 9 x 1000 samples array with
0s for now, reset to all zeros after restart program
volatile int recording_length[9] = {0}; // Array of the recording lengths of
each of the keys (when recorded and then played back)

// ADDED: COMPOSE MODE
volatile int compose_keys[MAX_SAMPLES] = {0}; // Make an array 1 x 1000 samples (for
now) and load with 0s

// ##### END AI-ASSISTED CODE #####
// Index and Keycode used to traverse through array
volatile int keycode;

// SPI data
uint16_t DAC_data; // output value

// DAC parameters
// A-channel, 1x, active
#define DAC_config_chan_A 0b0011000000000000
// B-channel, 1x, active
#define DAC_config_chan_B 0b1011000000000000

// SPI configurations
#define PIN_MISO 4
#define PIN_CS 5
#define PIN_SCK 6
#define PIN_MOSI 7
#define SPI_PORT spi0

```

```

// GPIO for timing the ISR
#define ISR_GPIO 2

// DDS sine table
#define sine_table_size 256
volatile int sin_table[sine_table_size];

// Keypad pin configurations
#define BASE_KEYPAD_PIN 9
#define KEYROWS 4
#define NUMKEYS 12

#define LED 25

unsigned int keycodes[NUMKEYS] = {0x57, 0x6E, 0x5E, 0x3E, 0x6D,
                                0x5D, 0x3D, 0x6B, 0x5B, 0x3B,
                                0x67, 0x37};

unsigned int scancodes[KEYROWS] = {0xE, 0xD, 0xB, 0x7};
unsigned int button = 0x70;

char keytext[40];
int prev_key = 0;

// ADDED in names for states for readability
// ##### BEGIN AI-ASSISTED CODE #####
typedef enum
{
    STATE_NOT_PRESSED,          // 0
    STATE_MAYBE_PRESSED,        // 1
    STATE_PRESSED,              // 2
    STATE_MAYBE_NOT_PRESSED     // 3
} state_keypad;

// ##### END AI-ASSISTED CODE #####
// Adding a function that does the scanning so we can just worry about the state
machine in the p-thread (just call)
// This function will return the keycode of the pressed button or -1 if
invalid/no-press, make void since no input params

int scan_keypad(void)
{

```

```

int i;
uint32_t keypad;

// Scan the keypad!
for (i = 0; i < KEYROWS; i++)
{
    // Set a row high
    gpio_put_masked((0xF << BASE_KEYPAD_PIN),
                    (scancodes[i] << BASE_KEYPAD_PIN));

    // Small delay required
    sleep_us(1);

    // Read the keycode
    keypad = ((gpio_get_all() >> BASE_KEYPAD_PIN) & 0x7F);

    // Break if button(s) are pressed
    if ((~keypad) & button)
        break;
}

// ##### BEGIN AI-ASSISTED CODE #####
// If we found a button . . . (EDITED)
if ((~keypad) & button)
{
    // Look for a valid keycode.
    for (i = 0; i < NUMKEYS; i++)
    {
        if (keypad == keycodes[i])
        {
            return i; // Return the keycode of the pressed button
        }
    }
}

return -1; // Return -1 if no valid keycode is found
}

// ##### END AI-ASSISTED CODE #####

// Alarm ISR
static void alarm_irq(void)
{
    // Assert a GPIO when we enter the interrupt

```

```

    gpio_put(ISR_GPIO, 1);

    // Clear the alarm irq
    hw_clear_bits(&timer_hw->intr, 1u << ALARM_NUM);

    // Reset the alarm register
    timer_hw->alarm[ALARM_NUM] = timer_hw->timerawl + DELAY;

    // DDS phase and sine table lookup
    phase_accum_main += phase_incr_main;
    DAC_data = (DAC_config_chan_B | ((sin_table[phase_accum_main >> 24] + 2048) &
0xffff)); // changed to channel B

    // Perform an SPI transaction
    spi_writel6_blocking(SPI_PORT, &DAC_data, 1);

    // De-assert the GPIO when we leave the interrupt
    gpio_put(ISR_GPIO, 0);
}

// =====
// === toggle25 thread
// =====
//
static PT_THREAD(protothread_toggle25(struct pt *pt))
{
    PT_BEGIN(pt);

    static unsigned int adc_val;

    // ##### BEGIN AI-ASSISTED CODE #####
    // More defined variables (added in)
    static state_keypad state = STATE_NOT_PRESSED; // Begin in the NOT_PRESSED state
and reset to it
    static int possible = -1; // Variable to hold the last valid
keycode
    static int phase_toggle = 1; // SET HIGH (WK 3 TONE GENERATOR)

    // Recording Variables
    static int record_rdy = 0; // Set high if press asterisk so on next cycle, can
see if previous cycle was asterisk

```

```

    static int record_flg = 0;    // Ready to enter record mode (set high if record rdy
(* is prev hit) and current key is 1-9 (STATE_PRESSED))
    static int record_key = -1;  // Can use keycode but for readability/access outside
STATE_PRESSED, can store the value of the key
    static int record_index = 0; // Current index to record at (should increment with
sampling rate) and store only freq

    // Playback Variables
    static int playback_flg = 0;    // Ready to enter playback mode (see in
MAYBE_PRESSED if keycode is 1-9 and record_rdy is 0, then set playback_flg high)
    static int playback_key = -1;  // Need to access outside of the loop (in ISR), can
store the value of the key
    static int playback_index = 0; // Current index to playback at (should increment
with sampling rate)

    // Compose Variables
    static int compose_flg = 0;      // Flag to indicate in compose mode (if low and
key pressed )
    static int compose_index = 0;    // Index to playback at (should increment after
another key press)
    static int compose_length = 0;   // Since need to be in STATE_MAYBE_PRESSED, keep
the length static and just a number
    static int compose_pb_flag = 0;  // Set a flag high once we press # again (set it
high if compose_flg == 1 and keycode == 11)
    static int compose_pb_index = 0; // Index for the pb part of compose (if less than
the compose length then go through and play the key presses in sequence)
    static int compose_next_key = -1; // Next key to get using playback_keys
    // ##### END AI-ASSISTED CODE #####

while (1)
{
    // toggle gpio 25
    gpio_put(LED_PIN, !gpio_get(LED_PIN));

    // Read the ADC
    adc_val = adc_read();
    curr_freq = adc_val * 2.5; // We want the actual frequency to be stored, not
the DDS phase increment (what we had previously)

    // ##### BEGIN AI-ASSISTED CODE #####
    // Call the keypad scan funtion
    // FSM ADDED FOR KEYPAD

```

```

        keycode = scan_keypad();
        printf("\nScanned Key: %d, | Old (Current) State: %d", keycode, state); // To
confirm scan_keypad outputs and current state of FSM

        switch (state)
        {
        case STATE_NOT_PRESSED:
            if (keycode == -1)
            {
                state = STATE_NOT_PRESSED; // Still not pressed, stay in NOT_PRESSED
state
            }
            else if (keycode != -1)
            {
                state = STATE_MAYBE_PRESSED; // Potentially pressed but not sure yet,
go to MAYBE_PRESSED state
                possible = keycode;           // Update the possible keycode (not sure
if bounce input yet) to be the input
            }
            break; // Default break for the NOT_PRESSED state (should thorhetically
never get here)

        case STATE_MAYBE_PRESSED:
            if (keycode == possible)
            {
                // ZERO KEYCODE LOGIC
                if (keycode == 0) // Press/Release 0 emit tone logic (2nd bullet point
Week2)
                {
                    if (phase_toggle == 1)
                    {
                        phase_toggle = 0;
                    }
                    else
                    {
                        phase_toggle = 1;
                    }
                }

                // ENTER/READY FOR RECORD MODE (PRESS * (10)) SET UP LOGIC
            }
            else if (keycode == 10)
            {

```

```

        record_rdy = 1;
        playback_flg = 0; // Set playback flag low so when press 1-9 enter
record mode and not play back freqs in array
        phase_toggle = 0; // TURN OFF THE PHASE TOGGLE SO NO SOUND HEARD
WHEN RECORDING

        printf("Ready to Record\n");
    }

    // ENTER RECORD MODE (PRESS 1-9) LOGIC
    else if ((keycode > 0) && (keycode < 10) && (record_rdy == 1))
    {
        record_flg = 1; // Set record flag high to enter record mode in
STATE_PRESSED

        playback_flg = 0; // Set playback flag low to not enter playback
mode in STATE_PRESSED (just to confirm)

        record_key = keycode; // Store the keycode of the pressed button to
use in STATE_PRESSED

        record_index = 0; // Reset the index to 0 to start recording at
the beginning of the array of the record_key-1th row

        recording_length[record_key - 1] = 0; // Reset the recording length
for the record_key-1th row to 0 to start recording at the beginning of the array of
the record_key-1th row
    }

    // ENTER COMPOSE MODE (PRESS # (11)) LOGIC
    // Enter Compose Mode
    else if ((keycode == 11) && (compose_flg == 0))
    {
        compose_flg = 1; // If we press # and the compose_flg is LOW, then
we will start recording the keys pressed until hit again (keycode = 11 and the flag is
HIGH, then set the flag LOW)

        compose_pb_flag = 0; // Not trying to playback anymore so set low

        compose_index = 0; // Reset the index for recording and the length
of the recording back to 0
        compose_length = 0;
    }

    else if ((keycode == 11) && (compose_flg == 1))

```

```

        {
            compose_pb_flag = 1; // Indicates that we've already pressed # in
the past and have recorded something, so now pressing again we set a flag high to
indicate playback

            compose_flg = 0; // Set back to low as no longer recording
            compose_pb_index = 0; // Ensure that compose_pb_index is reset

            playback_flg = 0; // Set the normal playback flag low (in compose
pb, not actual pb)
            playback_key = -1;
            playback_index = 0;
        }

        // Now that we've previously pressed # and are trying to record...
        else if ((keycode > 0) && (keycode < 10) && (compose_flg == 1)) //
Record the keys after keycode == 11 (as indicated by the flags being high)
            if (compose_index < MAX_SAMPLES)
            {
                compose_keys[compose_index] = keycode; // Get the keycode and
add it to the compose_keys array, then index
                compose_index++;

                compose_length = compose_index; // If last key press we record
it every cycle just in case
            }

            state = STATE_PRESSED; // Stable press confirmed, got to PRESSED state
        }

        else if (keycode != possible)
        {
            state = STATE_NOT_PRESSED; // False alarm, go back to NOT_PRESSED state
(no press/invalid)
            possible = -1; // Reset possible back to -1
(invalid/unstable press)
        }

        break; // Default break for the NOT_PRESSED state (should thorhetically
never get here)

    case STATE_PRESSED:
        if (keycode == possible)

```

```

        {
            if (((keycode > 0) && (keycode < 10)) && (record_flg == 1)) // If the
key is in 1-9 and the record flag is high...
            {
                if (record_index < MAX_SAMPLES) // If the value of the record index
is NOT exceeding the max samples then at record_key -1th row of array, store curr_freq
at record_index then iterate
                {
                    stored_freq[record_key - 1][record_index] = curr_freq;
                    record_index++;
                    printf("RECORD MODE | Recording freq: %d at Index: %d for Key:
%d\n", curr_freq, record_index, record_key);
                }
            }

            state = STATE_PRESSED; // Still being held
        }
        else if (keycode != possible)
        {
            state = STATE_MAYBE_NOT_PRESSED; // Potentially released but not sure
yet, go to MAYBE_NOT_PRESSED state
        }
        break; // Default break for the NOT_PRESSED state (should thorhetically
never get here)

    case STATE_MAYBE_NOT_PRESSED:
        if (keycode == possible)
        {
            state = STATE_PRESSED; // False alarm, go back to PRESSED state (still
being held)
        }
        else if (keycode != possible)
        {
            if (record_flg == 1)
            {
                recording_length[record_key - 1] = record_index; // Last index is
stored as the length for the pressed key
                record_flg = 0; // Reset the flag
                record_rdy = 0; // Reset the flag
                record_key = -1; // Reset back to
an invalid keycode
            }
        }
    }
}

```

```

        // ENTER PLAYBACK MODE (PRESS 1-9 but record_rdy is LOW and compose_flg
is NOT TRYING TO PLAYBACK) LOGIC
        else if ((possible > 0) && (possible < 10) && (record_rdy == 0) &&
(recording_length[possible - 1] > 0) && (compose_flg == 0)) // If the key is in 1-9
and the record_rdy flag is low and the recording length for the key is greater than 0
(meaning there is a recording to play back)...
        {
            playback_flg = 1;          // Set playback flag high to enter
playback mode in STATE_PRESSED
            playback_key = possible; // Store the keycode of the pressed button
to use outside
            playback_index = 0;        // Reset the index to 0 to start playing
back at the beginning of the array of the playback_key-1th row
        }

        state = STATE_NOT_PRESSED; // Stable release confirmed, go back to
NOT_PRESSED state
        possible = -1;              // Reset possible back to -1
(invalid/unstable press)
    }
    break; // Default break for the NOT_PRESSED state (should thorhetically
never get here)
}

// Playback Update
if ((playback_flg == 1) && (playback_index < recording_length[playback_key -
1]))
{
    curr_freq = stored_freq[playback_key - 1][playback_index]; // Got
playback_key back in STATE_MAYBE_PRESSED
    playback_index++;

    // printf("PLAYBACK MODE | Playing back freq: %d at Index: %d for Key:
%d\n", curr_freq, playback_index, playback_key);
}
else if ((playback_flg == 1) && (playback_index >=
recording_length[playback_key - 1]))
{
    playback_flg = 0;    // Reset the flag
    playback_key = -1;   // Reset back to an invalid keycode
}

```

```

        playback_index = 0; // Reset the index to 0 to start playing back at the
beginning of the array of the playback_key-1th row

        phase_incr_main = 0; // Reset the phase increment to 0 to stop the sound
from playing back after the last sample has been played back
        // phase_toggle = 0;    // TURNS OFF PHASE TOGGLE IF PREVIOUSLY ON (CHANGE
IF NEEDED)
    }

    // Compose Update
    if ((compose_pb_flag == 1) && (playback_flg == 0) && (compose_pb_index <
compose_length)) // If in compose mode and trying to record, and playback_flg (normal
pb) is LOW
    {
        // ##### END AI-ASSISTED CODE #####
        // ##### BEGIN AI-GENERATED CODE #####
        compose_next_key = compose_keys[compose_pb_index];
        // ##### END AI-GENERATED CODE #####
        // ##### BEGIN AI-ASSISTED CODE #####

        // Only play it if that key actually has a recording stored.
        if (recording_length[compose_next_key - 1] > 0)
        {
            // ##### END AI-ASSISTED CODE #####
            // ##### BEGIN AI-GENERATED CODE #####
            playback_key = compose_next_key;
            playback_index = 0;
            playback_flg = 1;
            // ##### END AI-GENERATED CODE #####
            // ##### BEGIN AI-ASSISTED CODE #####
        }

        compose_pb_index++;
    }

    // phase_incr_main Update rates (ZERO (phase_toggle = 1) VS ADC
VALUES/FUNCTIONS (phase_toggle = 0) LOGIC)
    if ((playback_flg == 1))
    {
        phase_incr_main = ((curr_freq * two32) / Fs); // Curr_freq comes from the
recording (ADC CASE)

```

```

        PT_YIELD_usec(1000); // 1 ms -> 1000 Hz playback
(10x speedup)
    }
    else if (record_flg == 1)
    {
        phase_incr_main = ((curr_freq * two32) / Fs); // Curr_freq comes from the
recording (ADC CASE)
        PT_YIELD_usec(10000); // 10 ms -> 100 Hz
normal/recording
    }

    else
    {
        phase_incr_main = ((curr_freq * two32) / Fs) * phase_toggle; // Previously
was adc_val * 2.5, but defined that as curr_freq
        PT_YIELD_usec(10000); // 10 ms ->
100 Hz normal/recording
    }

    // ##### END AI-ASSISTED CODE #####
    // printf(", New state: %d\n", state); COMMENTED OUT BECAUSE DELAYS THE SPEEDUP
    // printf("ADC value: %d\n", adc_val);
    // printf("Phase toggle: %d\n", phase_toggle);

} // END WHILE(1)
// every thread ends with PT_END(pt);
PT_END(pt);
} // end blink thread

int main()
{

    // Overclock (FROM KEYPAD)
    set_sys_clock_khz(150000, true);

    // Initialize stdio
    stdio_init_all();
    printf("Hello, DAC!\n");

    // Setup the ADC
    adc_init();
    adc_gpio_init(ADC_PIN);

```

```

adc_select_input(ADC_MUX);

// set up LED gpio 25
gpio_init(LED_PIN);
gpio_set_dir(LED_PIN, GPIO_OUT);
gpio_put(LED_PIN, true);

////////// KEYPAD INITS //////////
// Initialize the keypad GPIO's
gpio_init_mask((0x7F << BASE_KEYPAD_PIN));
gpio_set_dir((BASE_KEYPAD_PIN + 4), GPIO_IN);
gpio_set_dir((BASE_KEYPAD_PIN + 5), GPIO_IN);
gpio_set_dir((BASE_KEYPAD_PIN + 6), GPIO_IN);
// Set row-pins to output
gpio_set_dir_out_masked((0xF << BASE_KEYPAD_PIN));
// Set all output pins to low
gpio_put_masked((0xF << BASE_KEYPAD_PIN), (0xF << BASE_KEYPAD_PIN));
// Turn on pulldown resistors for column pins (on by default)
gpio_pull_up((BASE_KEYPAD_PIN + 4));
gpio_pull_up((BASE_KEYPAD_PIN + 5));
gpio_pull_up((BASE_KEYPAD_PIN + 6));

// Initialize SPI channel (channel, baud rate set to 20MHz)
spi_init(SPI_PORT, 20000000);
// Format (channel, data bits per transfer, polarity, phase, order)
spi_set_format(SPI_PORT, 16, 0, 0, 0);

// Setup the ISR-timing GPIO
gpio_init(ISR_GPIO);
gpio_set_dir(ISR_GPIO, GPIO_OUT);
gpio_put(ISR_GPIO, 0);

// Map SPI signals to GPIO ports
gpio_set_function(PIN_MISO, GPIO_FUNC_SPI);
gpio_set_function(PIN_SCK, GPIO_FUNC_SPI);
gpio_set_function(PIN_MOSI, GPIO_FUNC_SPI);
gpio_set_function(PIN_CS, GPIO_FUNC_SPI);

// === build the sine lookup table =====
// scaled to produce values between 0 and 4096
int ii;
for (ii = 0; ii < sine_table_size; ii++)

```

```

{
    sin_table[ii] = (int)(2047 * sin((float)ii * 6.283 / (float)sine_table_size));
}

// Enable the interrupt for the alarm (we're using Alarm 0)
hw_set_bits(&timer_hw->inte, 1u << ALARM_NUM);
// Associate an interrupt handler with the ALARM_IRQ
irq_set_exclusive_handler(ALARM_IRQ, alarm_irq);
// Enable the alarm interrupt
irq_set_enabled(ALARM_IRQ, true);
// Write the lower 32 bits of the target time to the alarm register, arming it.
timer_hw->alarm[ALARM_NUM] = timer_hw->timerawl + DELAY;

// === config threads =====
pt_add_thread(protothread_toggle25);

// === initialize the scheduler =====
pt_schedule_start;

// Nothing happening here
while (1)
{
}
return 0;
}

```